

Public Chat Portal for AlKhidmat

Kaavish Report
presented to the academic faculty
by

Syeda Samah Daniyal	sd07838
Rubab Shah	rs08104
Syeda Yashfeen Zehra Zaidi	sz08469
Fatima Dossa	fd08024



In partial fulfillment of the requirements for
Bachelor of Science
Computer Science

Dhanani School of Science and Engineering

Habib University
Spring 2026

Public Chat Portal for AlKhidmat

This Kaavish project was supervised by:

Hamas Malik
Digital Projects Lead
Al Khidmat Foundation

Ahsan Javed
Faculty of Computer Science
Habib University

Approved by the Faculty of Computer Science on _____.

Dedication

We sincerely dedicate this project to our beloved parents, who have been our greatest source of strength and support throughout our academic journey. Their constant love, endless prayers, and sacrifices have shaped us into the individuals we are today. We are truly grateful for their patience during the late nights and stressful deadlines of this final year.

We also dedicate this work to our siblings and friends for their continuous motivation and belief in our potential. A special dedication goes to our team, as this journey would not have been possible without our mutual respect and the countless hours we spent problem-solving together.

Finally, we dedicate this project to Almighty God for blessing us with the knowledge, health, and perseverance to complete this work. We are thankful for the guidance that allowed us to finish this project with purpose and dedication.

Acknowledgements

Our team would like to extend our heartfelt gratitude to our supervisor and mentors at Habib University for their constant guidance and technical oversight throughout this project. Their expertise and commitment to our learning allowed us to navigate various challenges and deliver a project we are proud of.

We express our sincere thanks to the Alkhidmat Karachi for providing us with the opportunity to work on this initiative. We are especially grateful to the domain experts and staff who shared their organizational knowledge and internal documentation, which formed the foundation of our knowledge base.

Abstract

The **Alkhidmat Public Chat Portal** is an AI-powered assistant built for one of Pakistan’s largest non-profit organisations, designed to automate the handling of daily inquiries from donors, volunteers, and beneficiaries across domains including healthcare, disaster relief, blood services, and orphan care.

The system implements an **Agentic Retrieval-Augmented Generation (RAG)** pipeline using OpenAI GPT-5 and Llama 3.2, with a six-stage self-validation critic that enforces factual grounding and suppresses hallucination. It supports multilingual interaction in **English, Urdu, and Roman Urdu**, processes image inputs such as medical receipts and payment documents via OCR, and routes low-confidence queries (below 0.70) to human agents through an automated escalation system. Role-based dashboards support ticket management and performance monitoring for staff and administrators.

The result is a scalable, multilingual portal that reduces manual query handling while maintaining response reliability — a deployable model for AI adoption in Pakistan’s non-profit sector.

Contents

1	Introduction	10
1.1	Problem Statement	10
1.2	Proposed Solution	10
1.3	Intended User	12
1.4	Project gantt chart and deliverables	13
1.5	Key Challenges	15
2	Literature Review	18
2.1	Introduction	18
2.2	State of the Art in RAG-Based Organizational Assistants	19
2.3	AI Chatbots in Nonprofits and Civil Society	20
2.4	Pakistan Context and Local-Language Chatbots	20
2.5	Similar Work in NGO Service and Donor Assistants	21
2.6	Multimodal Inputs and Accessibility	22
2.7	Human-in-the-Loop and Escalation Management	23
2.7.1	Uncertainty Estimation and Confidence Scoring in RAG Systems	24
2.8	Staff Coordination and Agent Dashboard	25
2.9	Analytics and Admin Tools	25
2.10	Cloud Deployment and Scalability	26
2.11	Security, Privacy, and Compliance	27
2.12	Differences Between Existing Work and the Proposed System	28
2.12.1	Multi-Domain NGO Scope Versus Single-Domain Chatbots . .	28
2.12.2	Deep RAG Over NGO-Specific Knowledge	28
2.12.3	Localization to Urdu, Roman Urdu, and Pakistani NGO Prac- tices	28
2.12.4	Integration With Operational Systems and Human Escalation	29
2.13	Conclusion	29

3	Software Requirement Specification (SRS)	30
3.1	Functional Requirements	30
3.2	Use Cases	37
3.3	Non-functional Requirements	37
3.4	System Diagram	40
3.4.1	Presentation Layer (Client-Side)	41
3.4.2	Application Layer (Backend)	41
3.4.3	AI/ML Layer	42
3.4.4	Data Layer (Storage and Retrieval)	42
3.4.5	Infrastructure Layer (Deployment)	42
3.4.6	Relationships Between Layers	43
3.4.7	Flow of Interaction	43
3.5	External Interfaces	44
3.5.1	User Interfaces - Wireframes	44
4	Software Design Specification (SDS)	49
4.1	Software Design	49
4.1.1	High-level Architecture	49
4.1.2	UML Class Diagram (PlantUML)	49
4.1.3	Classes and Services	50
4.2	Data Design	53
4.2.1	Entities, Attributes, Primary/Foreign Keys, and Cardinalities	54
4.2.2	Relational Schema (Core Tables)	55
4.2.3	Indexes and Search	55
4.2.4	Normalization and Cardinalities Rationale	55
4.2.5	Technical Details	56
4.2.6	Summary	58
5	Methodology, Experiments and Results	59
5.1	System Architecture: Agentic RAG and Frontend Integration	59
5.1.1	Frontend Architecture and User Interface	59
5.1.2	Agentic RAG Reasoning Engine	60
5.2	Experimental Setup	63
5.2.1	Dataset	63
5.2.2	Technical Configuration	63
5.2.3	QA Test Suite	64
5.3	Results and Performance Analysis	64
5.3.1	RAG Evaluation Metrics	64

5.3.2	Accuracy and Hallucination Control	66
5.3.3	Confidence Scoring and Escalation	67
5.3.4	Latency and Optimization	67
5.3.5	Multilingual and Multimodal Performance	68
6	Conclusion and Future Work	69
6.1	Conclusion	69
6.2	Future Work	69
7	Reflection	71
7.1	Reflections	71
7.1.1	Individual Reflections	71
7.1.2	Team Reflection	73
7.1.3	Process Reflection	73
7.1.4	Plans vs. Achievements	74
8	Data	76
	References	79

List of Figures

1.1	Month-Wise Deliverables	13
3.1	Use Case Diagram of the Public Chat Portal for Alkhidmat	37
3.2	High-level System Architecture	40
3.3	User Interface using Whatsapp	44
3.4	Staff Dashboard	45
3.5	Further Contents in Ticket Content Panel	46
3.6	Admin’s Dashboard - Analytics Monitor	47
3.7	Admin’s Dashboard - RAG Evaluation	48
4.1	UML Class Diagram showing the main system components, their relationships, and data entities	50
4.2	Relational schema diagram showing core tables, attributes, primary keys (underlined), and foreign key relationships	55
5.1	Hallucination comparison on an out-of-domain query. <i>Left:</i> Baseline LLM without the agentic layer generates an unsolicited, ungrounded response connecting weather to climate change in Pakistan. <i>Right:</i> Full pipeline with Router Agent active correctly classifies the query as out-of-domain and declines to generate.	61
5.2	RAG Evaluation Report generated by the Deepeval framework. Scores shown are for the English evaluation suite (10 cases). Answer Relevancy and Faithfulness both achieved 1.000; Contextual Precision scored 0.609; Contextual Recall scored 0.750; Hallucination scored 0.175 (lower is better). Overall pass rate: 6/8 eligible cases.	65
8.1	Data Pipeline Architecture	78

List of Tables

3.1	Non-Functional Requirements	38
5.1	Environment Specifications	63

1. Introduction

1.1 Problem Statement

Alkhidmat Foundation Pakistan, one of the largest non-profit organizations in the country, handles thousands of daily inquiries from donors, volunteers, and beneficiaries regarding donations, healthcare, and community welfare programs. Currently, these queries are managed manually through phone calls, emails, or in-person visits. This system leads to inefficiencies, delayed responses, redundant questions, and overburdened staff while also alienating overseas or low-literacy users as they struggle to navigate static website content or locate authentic project data

The lack of an automated communication solution restricts inclusivity and accessibility, especially for marginalized communities or overseas donors seeking quick, verified information. Moreover, Alkhidmat’s operations span multiple domains; healthcare (hospitals, pharmacies, diagnostics, blood banks), education, disaster management, orphan care, women empowerment, and more, creating a vast and complex information network that manual systems cannot efficiently support.

Hence, the problem is twofold:

1. Operational burden due to repetitive, manually handled queries.
2. Limited accessibility for donors and beneficiaries due to language, literacy, and technology barriers.

The motivation behind this project is to bridge this gap by creating an AI-driven, multilingual, multimodal public chat portal that ensures inclusivity, transparency, and efficient information flow which empowers both users and staff.

1.2 Proposed Solution

The proposed solution is an AI-powered Public Chat Portal for Alkhidmat Foundation, a unified platform integrating a Retrieval-Augmented Generation (RAG)-based

LLM with multimodal capabilities (text and image input). The portal will be accessible through Alkhidmat’s official website and WhatsApp Business API to allow seamless communication across all major user groups; donors, patients, blood donors, and beneficiaries.

System Overview:

1. **Frontend Chat Interface:** Accessible through two primary channels:
 - **WhatsApp Interface:** Integrated via the WhatsApp Business API, allowing users to chat directly through WhatsApp using text, voice notes, and image uploads. The interface employs structured message templates and quick-reply buttons for guided workflows, ensuring inclusivity for overseas donors and non-technical users.
 - **Web Interface:** Built in React for responsive and multilingual interaction (English, Urdu, Roman Urdu), providing a modern and intuitive experience for both desktop and mobile users.
2. **Backend Services:** Powered by FastAPI and LangChain, the backend manages query classification, retrieval, and response generation. It includes:
 - **Intent Classification Module:** Determines the user’s domain of inquiry (e.g., Donation, Healthcare, Blood Services, General Support).
 - **Retrieval-Augmented Generation (RAG) Engine:** Combines LLM reasoning with a verified, domain-specific knowledge base to produce contextually accurate and concise responses.
 - **Multimodal Processing Layer:** Handles image understanding and context management for smooth multimodal input handling.
 - **Security and Compliance Layer:** Enforces end-to-end encryption, anonymization of user data, and access control for backend APIs.
3. **Knowledge Base (RAG Datasets):** Curated and periodically updated datasets covering:
 - Donation and payment workflows
 - Healthcare and welfare program details
 - Donor engagement and transparency data
 - CSR and corporate partnership information
 - Accessibility and support resources

4. **Human Escalation Dashboard:** A staff-facing interface where unresolved or sensitive queries are routed to human agents. Staff can respond directly, monitor conversation histories, and update relevant dataset entries.
5. **Analytics and Monitoring Module:** Tracks user engagement, response accuracy, and escalation frequency. Insights from these analytics help refine the system’s knowledge base and improve LLM performance over time.

1.3 Intended User

The chat portal caters to three primary user groups, identified through persona analysis from workshops with Alkhidmat:

1. **Donors (Local, Overseas, Corporate):**

- Seek transparency, convenience, and verified information about donation channels, project impact, and tax compliance.
- Subtypes include recurring donors, new donors, first-time donors, and CSR/corporate donors.

2. **Healthcare Beneficiaries (Patients, Blood Donors, Recipients):**

- Require guidance on healthcare services, test prices, facility locations, doctor availability, ambulance booking, and Zakat-based assistance.
- Includes rural and urban patients with varying digital literacy levels.

3. **General Users (Public Information Seekers):** Represent individuals who approach Alkhidmat for general inquiries such as volunteer opportunities, branch contact details, upcoming drives, disaster relief information, and awareness campaigns. The chatbot provides them with general updates, project overviews, and links to participation or registration forms.

Through intent classification, each user’s domain (donation, healthcare, general inquiry) is automatically detected, ensuring relevant responses and routing. Administrative users (Alkhidmat staff) access a dashboard to manage escalations, update datasets, and broadcast campaigns via WhatsApp APIs.

1.4 Project gantt chart and deliverables

		YEAR 1				YEAR 2				End of project
	Oct 2025	Nov 2025	Dec 2025	Jan 2026	Feb 2026	March 2026	April 2026			
	Requirements approval	Design completion	Prototype ready	Human routing operational	Full integration	Deployed beta	Final defence			
Phase / Task	Phase 1 – Project Initiation & Requirements Gathering	System Design & Data Preparation	Prototype Development – Intent Classifier & RAG Setup	Phase 2 – Human Dashboard & Escalation Routing	Integration & Testing	Analytics Module & Cloud Deployment	Final Testing, Documentation & Presentation			
Deliverables & Milestones	- Requirements document - Domain map (12 domains) - Alkhidmat User flow & chatbot conversation map	- UX wireframes for chat interface - Dataset collection & cleaning from Alkhidmat resources - Initial ER diagram and API routes	- Trained ML classifier (Intent detection) - Integrated RAG pipeline using LangChain + Llama 2 - Basic chatbot interface connected to backend	- React dashboard for staff queries - Escalation logic & workflow integration - Internal test report (AI ↔ Human routing)	- Full system integration (chat portal + dashboard + RAG) - Unit & integration tests - Initial usability feedback	- Analytics dashboard (query trends, escalation rates) - Dockerized deployment (Heroku) - Beta version live for demo	- Final project report & documentation - User guide & demo video - Final FYP presentation and viva			
Responsible Members	All members (collaboration) Lead: Samah (documentation structure), Fatima (domain mapping)	1.Fatima: UX wireframes 2.Samah: Database schema & API design 3.Yashfeen: Data preprocessing 4.Rubab: Data logging plan	1.Yashfeen: Classifier model 2.Fatima: Frontend integration 3.Samah: Backend FastAPI endpoints 4.Rubab: Data logs for test runs	1.Samah: Backend logic & Dashboard APIs 2.Rubab: Dashboard UI/UX 3.Fatima: Chatbot ↔ dashboard connection 4.Yashfeen: Confidence-based routing model	1.Samah: Backend & deployment tests 2.Yashfeen: Model evaluation 3.Rubab: Usability test reports 4.Fatima: Frontend bug fixes	1.Rubab: Analytics module 2.Samah: Docker + Heroku deployment 3. Fatima: UI enhancements 4. Yashfeen: Data validation metrics	All members (joint effort) 1.Samah: Technical report 2.Rubab: Visuals & analytics 3.Fatima: Presentation slides 4. Yashfeen: Model performance summary			
Deliverable Output	Project charter, domain map, workflow	UX wireframes, DB schema	Basic AI chatbot working	Dashboard connected	Working AI + Human chat flow	Live Dockerized version	Documentation & presentation			

Figure 1.1: Month-Wise Deliverables

Deliverables Overview

The Gantt chart above outlines the project objectives, key milestones, and timelines for each phase across Kaavish I and Kaavish II. Major deliverables include:

- **Kaavish I:** System requirements analysis, dataset design, RAG knowledge base creation, chatbot workflow design, and prototype implementation of the frontend and backend communication pipeline.
- **Kaavish II:** Integration of AI/ML modules (intent classification, multimodal processing), deployment on cloud infrastructure (Heroku/Docker), human escalation dashboard, and final evaluation with Alkhidmat's stakeholders.

Resources Needed

While the Gantt chart defines tasks, timelines, and team responsibilities, successful execution also depends on the following essential resources:

1. Hardware and Computing Resources

- Developer laptops with minimum 16GB RAM and GPU support for model inference.
- Cloud hosting credits or Heroku dynos for deployment and load testing.
- External SSDs for backup of datasets and logs.

2. Software and Development Tools

- **Backend:** FastAPI, Python, LangChain, and Llama 2 integration.
- **Frontend:** React.js, Tailwind CSS, and WhatsApp Business API integration.
- **Database and Storage:** PostgreSQL, Elasticsearch, and Firebase (for chat logs and persona data).
- **Version Control and CI/CD:** GitHub, Docker, and Heroku pipelines.

3. AI/ML and Data Resources

- Access to pretrained LLMs (e.g., Llama 2 or OpenAI API for evaluation).
- Dataset curation support from Alkhidmat domain experts for healthcare, donation, and welfare programs.

- Preprocessing tools for Roman Urdu and Urdu text normalization.

4. Human Resources

- **Development Team:** 4 members (Frontend, Backend, NLP, and Integration roles).
- **Supervisors:** Internal advisor for technical oversight; external industry mentor from Alkhidmat.
- **Domain Experts:** Staff providing verified data and validating chatbot responses.
- **Test Users:** Donors, patients, and volunteers for pilot testing and feedback collection.

5. Other Resources

- Secure API keys for WhatsApp Business and cloud hosting.
- Access to Alkhidmat’s internal documentation, SOPs, and donation workflow data.
- Dataset annotation tools for custom tagging of healthcare and donation intents.

These resources collectively ensure that each phase of development outlined in the Gantt chart is feasible and aligned with the technical and operational goals of the project.

1.5 Key Challenges

This section mentions the key challenges that we foresee in this project and possible ways to address them.

1. Data Acquisition and Knowledge Base Consolidation (Pre-deployment)

- **Challenge:** Acquiring data and building a reliable RAG knowledge base is an immediate challenge since Al-Khidmat does not currently have a single, consolidated, structured database. This requires extensive web scraping and manual formatting of disparate documents across multiple domains, which is complicated by time constraints and the need for domain expertise.

- **Mitigation:** This challenge has been explicitly communicated to the external supervisor. The team has prioritized the most impactful datasets through user personas (e.g., Blood Donation SOPs). The mitigation strategy involves assigning a dedicated industry partner resource to gather and provide this essential, formatted data ASAP, aiming for the core datasets to be compiled within the next 1–2 weeks.

2. Multilingual and Roman Urdu Accuracy (Natural Language Processing)

- **Challenge:** Achieving high accuracy in Urdu and handling the inconsistency of Roman Urdu (Urdu written in English script) is difficult, as LLMs often lack high fidelity for low-resource languages and phonetic variations. This directly impacts the $\geq 90\%$ accuracy goal.
- **Mitigation:** Dedicate significant effort to curating the Urdu component of the RAG knowledge base. Employ custom preprocessing steps to normalize common Roman Urdu spellings before feeding them to the LLM/RAG engine.

3. RAG Data Volatility and Latency (Data Management)

- **Challenge:** Al-Khidmat’s program details (e.g., active disaster relief efforts, current blood stock) change frequently. Since the system relies on a **periodically updated** knowledge base, there is a risk of providing outdated information, which compromises user trust and data integrity.
- **Mitigation:** Establish clear and enforced SLA for Admin updates to the RAG knowledge base (e.g., daily update requirement for time-sensitive domains like Blood Services). Use the Human Escalation Dashboard to flag and correct critical data mismatches immediately.

4. Balancing Transparency and Privacy (Security and Ethics)

- **Challenge:** Donors seek transparency about fund utilization, but the system must rigorously protect beneficiary and personal data (e.g., Zakat recipient details, patient health information) while maintaining local compliance standards.
- **Mitigation:** Implement stringent anonymization/pseudonymization for all data logs. Use guardrails to strictly prevent the LLM from generating or retrieving specific identifiable information in public responses.

5. Human Handoff and Agent Training (Operational/UX)

- **Challenge:** Ensuring a seamless handover from AI to a human agent without frustrating the user, particularly on WhatsApp where the 24-hour window constraint applies. Agents also need specialized training to use the Human Escalation Dashboard efficiently (diagnosing the bot's failure, not just the user's query).
- **Mitigation:** Focus on quick context loading and pre-filled response templates in the dashboard. Develop mandatory training modules for agents specifically on "AI Failure Diagnosis" and persona-aware communication.

2. Literature Review

2.1 Introduction

The rapid advancement of artificial intelligence (AI), particularly large language models (LLMs), has significantly reshaped organizational knowledge management and service delivery. Retrieval-Augmented Generation (RAG) enhances LLMs by grounding responses in external knowledge sources, improving factual accuracy, reducing hallucinations, and increasing transparency [20, 10, 7]. In parallel, nonprofit and humanitarian organizations are increasingly adopting conversational systems for service navigation, donor engagement, and community outreach [8, 28, 24].

This literature review synthesizes research across four interconnected areas to inform the design of a multi-domain organizational assistant for a Pakistani NGO. First, it examines RAG-based conversational systems, including advances in persona adaptation, multi-turn dialogue, and multimodal interaction. Second, it reviews chatbot deployments for social good, with a focus on low- and middle-income country (LMIC) contexts and local-language systems in Urdu and Roman Urdu. Third, it explores system-level considerations, including human-in-the-loop escalation, operational dashboards, and analytics. Finally, it considers infrastructure, security, and deployment practices required for scalable and reliable systems.

A key gap emerges from this synthesis: existing chatbot systems in Pakistan are predominantly single-domain and loosely integrated with organizational workflows. There is limited evidence of a unified, multi-domain RAG-based assistant capable of supporting donations, healthcare services, blood programs, and welfare initiatives across English, Urdu, and Roman Urdu. The proposed Alkhidmat Public Chat Portal addresses this gap by combining RAG grounding, multilingual interaction, multimodal inputs, confidence-aware escalation, and integrated operational tooling within a single system.

2.2 State of the Art in RAG-Based Organizational Assistants

For tasks that require a lot of knowledge, Retrieval-Augmented Generation (RAG) was introduced as a general framework that combines pretrained sequence to sequence models with a non-parametric memory of retrieved documents [20]. Lewis et al. note that models augmented with retrieval not only generate more accurate and factual responses but also outperform both purely parametric models and traditional retrieve-and-extract approaches in open-domain question answering.

The RAG framework has since been organised and made clearer by a number of survey studies. A thorough analysis of RAG in large language models is given by Gao et al. [10], who also discuss naive, advanced, and modular architectures and explain how the retrieval, generation, and augmentation components interact. Similar to this, Fan et al. concentrate on Retrieval-Augmented Large Language Models (RA-LLMs), looking at their architectures, training methods, and uses in domains like recommendation systems and customer service [7]. When taken as a whole, these surveys show that RAG has emerged as a crucial strategy for producing organisational assistants who are trustworthy and able to maintain a solid foundation in current, authoritative knowledge.

At the same time, research on conversational AI has made significant progress in areas such as persona modeling, multi-turn context tracking, and socially aware interactions. Surveys of neural conversational agents show a clear shift from simple, single-turn question answering toward multi-turn, task-oriented, and open-domain dialogue systems that can adapt both style and content to meet users' needs [17]. Persona-aware dialogue models take this a step further, adjusting tone and content based on user or role profiles—an important consideration when designing assistants for beneficiaries, donors, or staff who may have different information requirements [29].

From these trends, contemporary organizational assistants typically:

- use RAG to ground answers in internal knowledge bases;
- support multi-channel access (web, mobile, messaging apps);
- adapt to different user personas and roles;
- include monitoring, guardrails, and human-in-the-loop escalation.

The proposed Alkhidmat assistant follows this pattern by coupling an LLM front-end with domain-structured knowledge on donations, healthcare, blood services, and welfare programmes, plus escalation and analytics for staff.

2.3 AI Chatbots in Nonprofits and Civil Society

While RAG provides the technical foundation for knowledge-grounded systems, its application in real-world settings—particularly in the nonprofit sector—introduces additional design constraints related to accessibility, trust, and inclusion.

Within the nonprofit and social-impact space, there is growing interest in deploying chatbots for information provision, advocacy and service navigation. Følstad et al. discuss “chatbots for social good” as a research and design agenda, identifying use cases in health, education, civic participation and humanitarian response, and highlighting issues of inclusion, trust, and governance [8]. They note that many early systems are narrowly scoped, relying on scripted or intent-based flows.

Empirical work on health-related chatbots in low and middle income countries (LMICs) provides concrete examples. Munir et al. report on the feasibility of an “artificially intelligent vaccines chatbot” (Bablibot) in Pakistan, deployed via text messaging to provide immunization information to caregivers in low-income communities [24]. Their mixed-methods evaluation finds high user satisfaction and suggests that chatbots can address access barriers, especially for women. Building on this, Siddiqi et al. present a detailed mixed-methods study of Bablibot, describing its design using natural language processing (NLP), machine learning and a human-in-the-loop component, and show that local-language chatbots are feasible and acceptable in low-resource, low-literacy settings [28].

Beyond health, chatbots are increasingly used for fundraising and advocacy by nonprofit organizations, including for storytelling and campaign communication on platforms such as Facebook Messenger [13]. However, many of these systems focus on a single function (e.g., advocacy narratives, fundraising journeys) and are not deeply integrated with internal programme databases or multi-department service structures.

2.4 Pakistan Context and Local-Language Chatbots

These challenges are further amplified in the Pakistani context, where linguistic diversity and resource constraints shape the design and deployment of conversational systems. Pakistan is an important context for AI-mediated services because it is a lower middle income, linguistically diverse country with high mobile and messaging usage. Rich morphology, complex script, and cross-script usage are some of the unique NLP challenges faced by Urdu and Roman Urdu. In their review of problems

and difficulties in Urdu Natural Language Processing, Khan et al. point out the lack of annotated tools and resources and identify tasks like tokenisation, morphological analysis, POS tagging, parsing, and named-entity recognition as ongoing research issues [16]. The development of Urdu NLP resources and applications is also described in a more comprehensive survey of Urdu language processing [3].

Several recent projects have focused on Urdu and Roman Urdu in conversational AI. For instance, Kumar et al. developed SAATHI, a voice-enabled Urdu virtual assistant designed to help senior citizens in Pakistan who are “ageing in place” by providing news, reminders, and simple interactions in Urdu [18]. In another project, Mohiuddin et al. created a multilingual Urdu chatbot using the Rasa framework, which tackles the challenge of converting Roman Urdu input into Urdu intents [23]. Similarly, Hakeem et al. describe an AI-powered chatbot for Roman Urdu, combining GPT-style models with a RAG framework to improve handling of spelling variations and the lack of standardization [12].

At the same time, evaluation of LLMs on Urdu remains an active area. Kazi et al. evaluate large language models on Urdu–English question answering, showing significant performance gaps between high-resource and low-resource settings and underlining the need for Urdu-specific adaptation and datasets [15]. These works collectively demonstrate that:

- Urdu and Roman Urdu chatbots are technically feasible;
- local-language assistants can be deployed in low-resource Pakistani communities;
- there is still limited work on large, multi-domain assistants tightly integrated with a single NGO’s internal systems.

2.5 Similar Work in NGO Service and Donor Assistants

Building on these contextual considerations, prior work on NGO-oriented assistants can be grouped into distinct categories based on domain scope and system capabilities. In the specific domain of nonprofit and NGO assistants, the most relevant academic work can be grouped into three strands.

First, chatbots for social good and humanitarian applications. The CHI extended abstract on chatbots for social good explicitly frames chatbots as tools for health, education and civic engagement, and calls for systems that can scale while

remaining inclusive and trustworthy [8]. This provides a conceptual foundation for NGO-oriented assistants such as the Alkhidmat portal.

Second, health and immunization chatbots in Pakistan. Munir et al. and Siddiqi et al. both focus on Bablibot, a local-language chatbot for immunization information in Karachi [24, 28]. Bablibot is tightly connected to immunization registries and demonstrates integration with digital health records, but it is single-domain (vaccines) and not a general NGO service portal.

Third, Urdu / Roman Urdu service chatbots and administrative bots. Mohiuddin et al. target multi-lingual service access via transliteration-based Urdu chatbots using Rasa [23]; Kumar et al. design SAATHI for elderly care information [18]; Hakeem et al. build a Roman Urdu chatbot using RAG for more open-ended conversational understanding [12]. Maqbool et al. propose a sentence-level encoder-decoder architecture for an intelligent administrative chatbot for Roman Urdu, designed to support university administrative tasks [21]. These systems are structurally closer to an organizational assistant, but they are focused on administrative or general chat tasks rather than multi-department NGO services.

There is little evidence in the academic literature of a *single* Pakistani NGO deploying a RAG-based assistant that simultaneously covers donations, healthcare navigation, blood services, welfare programmes, and volunteer opportunities across web and WhatsApp.

2.6 Multimodal Inputs and Accessibility

Modern conversational assistants increasingly handle not just text but multiple input modalities, such as images and voice, significantly enhancing accessibility and interaction.

Recent work on Multimodal RAG (Retrieval-Augmented Generation) shows how chatbots can analyze and reason over images, tables, and text together to enrich responses. For instance, systems can perform OCR or caption images and retrieve relevant text or table data to answer user queries, reflecting an alignment with research on "multimodal" chatbots that combine vision and language for richer dialogue [11].

State-of-the-art models, such as GPT-4V ("GPT-4 with Vision"), now support direct image input, processing raw image bytes alongside text [11]. OpenAI has added similar image and voice capabilities to ChatGPT, allowing users to "show ChatGPT what you're talking about" and converse by voice [25]. An AI chatbot might use convolutional neural networks (CNNs) to parse objects and context from an uploaded photo [1].

In practical terms, this means our chatbot can accept photos of payment receipts, prescriptions, or donation screenshots and extract contextual information from them [11].

Voice interfaces are also becoming common for accessibility:

- A speech-to-text (STT) or automatic speech recognition (ASR) component can transcribe user voice notes (with basic Urdu support) into the dialogue, allowing users who prefer speaking to interact with the bot [11], [1].
- In a Pakistani NGO context, this means a donor could send a voice note in Urdu that is transcribed and answered via ASR, bridging literacy and connectivity gaps [1], [25].

Integrating image and speech inputs enables the system to serve non-tech-savvy, low-literacy users, or those with disabilities, extending the chatbot beyond "plain text" to richer multimodal engagement [11]. Such features make digital assistants more inclusive (e.g., by letting low-literacy users show receipts or speak questions) [25].

These multimodal capabilities are rapidly dropping in cost and complexity as open-source vision models (e.g., CLIP, BLIP-2) and multimodal LLM APIs (e.g., GPT-4V, Claude Vision) become more accessible [11].

2.7 Human-in-the-Loop and Escalation Management

Industry best practices emphasize that chatbots should gracefully escalate complex or ambiguous queries to human agents to maintain user trust and ensure effectiveness [9]. In hybrid AI support systems, escalation is typically triggered when the model's confidence is low or when the user's request falls outside the chatbot's defined domain, such as medical emergencies or detailed calculations. These systems rely on monitoring intent confidence and transferring any ambiguous or sensitive query to staff to prevent misinformation and ensure reliability [9]. Platforms such as WATI, a WhatsApp Business API provider, operationalize this principle by implementing hybrid workflows in which the AI bot "automatically answers common queries" but "routes more complex queries to your support teams/agents based on customer intent" [32]. In addition, industry vendors stress that human agents must retain complete control to manage AI conversations manually when required, ensuring operational oversight, accountability, and continuity in user experience.

2.7.1 Uncertainty Estimation and Confidence Scoring in RAG Systems

A key challenge in implementing human-in-the-loop systems is determining *when* escalation should occur. This requires explicit estimation of model confidence and uncertainty, which has become an active area of research in LLM-based systems. Reliable deployment of RAG-based assistants requires explicit modeling of uncertainty to decide when to answer, abstain, or escalate. Recent work categorizes confidence estimation methods into four classes: token-level, self-verbalized, semantic similarity-based, and mechanistic approaches [31].

Token-level methods leverage the probabilistic nature of LLMs. Since models generate tokens sequentially with associated probabilities, confidence can be derived from log-probabilities. Common techniques include:

- *Average log-probability* and *maximum token uncertainty* for sequence-level confidence;
- *Perplexity*, where higher values indicate greater uncertainty due to a flatter probability distribution;
- *Entropy-based scoring*, where low entropy reflects confident (peaked) token distributions and high entropy indicates uncertainty [31].

Recent applied approaches propose weighted confidence formulations that emphasize high-certainty tokens. For example, combining the joint probability of top-ranked tokens with overall sequence probability (e.g., 70/30 weighting) has been shown to stabilize confidence estimates in generation tasks [30].

Sampling-based methods, such as SelfCheckGPT, estimate confidence by generating multiple responses and measuring consistency across them. Higher agreement across sampled outputs implies higher confidence, making this approach suitable for black-box models where logits are unavailable, albeit at higher computational cost [19].

Self-verbalized confidence asks the model to report its own certainty. While simple, this approach is prone to overconfidence and is typically used in combination with other signals [19].

Retrieval-aware confidence is also critical in RAG systems. A common proxy is the semantic similarity between the query and retrieved documents (e.g., cosine similarity), aggregated across top- k chunks to estimate retrieval reliability [6]. This complements generation-side confidence.

Threshold selection plays a central role in operationalizing confidence. Industry and research suggest tuning thresholds empirically on validation data to optimize

correctness, often by sweeping values between 0 and 1 and selecting those that maximize accuracy for high-confidence responses [27, 26]. These thresholds directly inform fallback strategies such as abstention or human escalation.

Implication for system design: In the proposed system, confidence is modeled as a hybrid signal combining (i) token-level generation confidence, (ii) retrieval similarity scores, and (iii) optional sampling-based validation for critical queries. This composite score governs response acceptance, rejection (“I don’t have enough information”), and routing to human agents, aligning with best practices for trustworthy AI deployment.

2.8 Staff Coordination and Agent Dashboard

The operationalization of human-in-the-loop handoff is facilitated through shared agent dashboards, often referred to as “team inboxes,” as exemplified by platforms such as WATI [32]. These dashboards are designed to enable seamless coordination among staff when managing escalated queries. Each escalated chat is presented as a ticket, allowing agents to access the complete conversation history and respond directly through the same channel, including bilingual responses where required [32]. Domain-based routing capabilities are commonly integrated, ensuring that specific queries, such as those related to donations, are directed to the appropriate team (e.g., fundraising) and can be dynamically reassigned among specialists. The inclusion of pre-approved templates supports standardized and high-quality responses, particularly within platform-specific constraints such as WhatsApp’s 24-hour reply window, while the logging of agent actions provides traceability and auditability. Furthermore, these systems are designed to maintain continuous availability and multilingual support, enabling consistent handoff and follow-up interactions irrespective of whether the user communicates in English, Urdu, or Roman Urdu. Vendors highlight that such implementations provide “24/7 automated support that responds naturally and accurately in your customers’ preferred language” [32], thereby ensuring both operational efficiency and user satisfaction.

2.9 Analytics and Admin Tools

Effective deployments of organizational chatbots require comprehensive analytics and content management to guide continuous improvement, maintain accuracy, and ensure transparency over time. Performance monitoring is typically conducted through analytics dashboards that report Key Performance Indicators (KPIs) such as query

volume, total queries handled, and deflection rate, indicating the percentage of interactions resolved by the AI versus those escalated to human agents. Industry guides emphasize escalation or human-takeover rate as a vital KPI, noting that low takeover rates suggest strong autonomous handling, whereas spikes may signal knowledge gaps or safety concerns [9]. Additional time-based metrics commonly tracked include average response time and session duration, and systems routinely collect user satisfaction data, including CSAT (thumbs-up/down), to assess perceived quality [9]. Real-time dashboards also chart usage by domain (e.g., donations or health queries) and peak hours, enabling administrators to interpret logs, review conversation transcripts, and incorporate user feedback to iteratively improve the bot, refine intents or flows, and update the RAG knowledge base [9]. The literature consistently underscores that chatbots must function as “data-driven” tools [9].

A human-facing administrative interface complements analytics by enabling operational control and content management. Staff require the ability to manually update program information, such as donation campaigns and hospital details, and maintain the knowledge repository. In the context of a Pakistani NGO, such an interface would also allow editing of Urdu translations, defining personas, and monitoring escalation tickets in real time. Administrative configuration capabilities typically include adjusting fallback messages, modifying quick-reply templates, and setting content moderation rules. Systems comparable to enterprise chat platforms incorporate “team inbox” operational views alongside update forms for knowledge base maintenance [32]. Together, automated analytics and manual administrative controls align with recognized best practices in RAG-based assistant deployment, ensuring sustained accuracy, transparency, and ongoing system improvement.

2.10 Cloud Deployment and Scalability

To accommodate large user volumes, ensure high availability, and facilitate rapid growth, conversational platforms are typically deployed on cloud infrastructure with containerized architectures. Many organizations, including non-governmental organizations (NGOs), leverage Platform-as-a-Service (PaaS) solutions such as Heroku, AWS, and Google Cloud, which provide autoscaling capabilities, managed databases, and high availability [14], [22]. Salesforce’s Live Agent chatbot, for instance, is deployed on Heroku, utilizing managed Postgres databases and benefiting from dedicated operational staff responsible for backups, updates, and outage management, thereby reducing the operational burden on developers [14]. Heroku advertises its PaaS offering as providing “fully managed infrastructure... with built-in autoscaling

and detailed performance metrics” in a “compliant environment.”

The use of PaaS or containerized cloud clusters ensures both high availability and seamless horizontal scalability, enabling the system to accommodate hundreds of simultaneous users and remain resilient during peak usage periods [14]. Managed cloud services further enhance operational efficiency by providing automated backups, secure key management, and CI/CD pipelines for version control, along with real-time performance monitoring [14], [22]. Technical components of such deployments often include Docker containers, and services like Elasticsearch or pgvector for Retrieval-Augmented Generation (RAG) indexing, which support maintainable and responsive chatbot systems. Leveraging these cloud-based practices enables platforms to reliably support large-scale user interactions while maintaining uptime exceeding 99%.

2.11 Security, Privacy, and Compliance

For NGOs deploying conversational agents, safeguarding user data is of paramount importance and must align with both industry standards and relevant legal frameworks. The architecture of such systems should integrate robust encryption mechanisms, ensuring that all communications, particularly those containing sensitive personal or medical information, are secured in transit and at rest. For web interactions, HTTPS/TLS protocols should be applied, while WhatsApp-based interactions benefit from end-to-end Signal encryption inherent to the platform [33].

Beyond technical security, adherence to global compliance standards is critical. The WhatsApp Business Platform holds certifications including SOC 2 and explicitly supports GDPR compliance, providing a framework for lawful data processing [33]. In addition, organizations operating in Pakistan should comply with emerging national regulations, such as the draft Personal Data Protection Bill (2023), which mirrors GDPR by mandating user consent, data minimization, and breach accountability [5].

Data minimization policies are central to ethical and legal compliance. Best practices include obtaining explicit user consent [33], [5], retaining only transient session logs, and deleting chat histories after completion of interactions [33]. Personal identifiers should be anonymized, and sensitive personally identifiable information (PII) should not be stored beyond operational necessity [5].

In conclusion, a secure and compliant chatbot deployment integrates the built-in end-to-end encryption and certifications of the WhatsApp Business API while implementing internal policies for data minimization and local regulatory compliance.

This combination ensures both technical security and adherence to legal and ethical standards, providing a reliable framework for NGO services [33], [5].

2.12 Differences Between Existing Work and the Proposed System

Synthesizing the above, the proposed Public Chat Portal for Alkhidmat differs from existing work in several ways.

2.12.1 Multi-Domain NGO Scope Versus Single-Domain Chatbots

Existing Pakistani chatbots in the literature typically focus on one domain: immunization (Bablibot) [24, 28], elderly care information (SAATHI) [18], administrative tasks in education [21], or general Roman Urdu conversational understanding [12]. In contrast, the Alkhidmat assistant is explicitly designed as a multi-department organizational portal, covering donations (e.g., Zakat, Sadaqah), healthcare facilities, blood services (donation and requests), other general facilities.

2.12.2 Deep RAG Over NGO-Specific Knowledge

Most NGO-relevant chatbots described in the literature are intent or flow based, with relatively shallow use of organizational documents or databases [8, 23]. By contrast, the proposed system follows the RAG paradigm discussed by Lewis et al. and later surveys [20, 10, 7], building a structured knowledge base from Alkhidmat’s internal SOPs, programme descriptions, facility lists, and operational data, and using retrieval to ground responses. This is closer to enterprise RAG assistants but applied in a nonprofit setting.

2.12.3 Localization to Urdu, Roman Urdu, and Pakistani NGO Practices

While prior work has shown that Urdu and Roman Urdu chatbots are feasible [16, 18, 23, 12], these systems are typically either domain-specific (health, elderly care) or experimental. The Alkhidmat assistant aims to:

- support English, Urdu, and basic Roman Urdu input;

- incorporate programme names and concepts specific to a large Pakistani NGO (e.g., Mawakhat, Orphan Care, Bano Qabil);
- operate on channels (web and WhatsApp Business) that match actual user behaviour in Pakistan.

This combination of linguistic and institutional localization is not extensively covered in existing academic work.

2.12.4 Integration With Operational Systems and Human Escalation

The Bablibot work demonstrates integration with immunization registries and a human-in-the-loop support model [28, 24]. The proposed Alkhidmat portal extends this logic beyond health to other NGO operations, for example:

- a blood-donation microservice that can check donor eligibility criteria and facility locations;
- dashboards where staff can view escalated conversations with context and respond directly;
- analytics on domain-wise query volumes, response quality, and escalation rates.

This reflects best practices in RAG-based enterprise assistants (monitoring, feedback loops) but adapted for NGO governance. Escalation decisions are typically driven by confidence thresholds derived from uncertainty estimation. Queries with low generation confidence or weak retrieval grounding are automatically routed to human agents, ensuring reliability and preventing hallucinated responses [27, 26].

2.13 Conclusion

The literature establishes Retrieval-Augmented Generation as a robust framework for building grounded and reliable conversational systems. At the same time, research in conversational AI emphasizes the importance of multi-turn interaction, persona adaptation, and user trust. In nonprofit contexts, chatbot deployments demonstrate feasibility and impact, particularly in LMIC settings, but remain largely narrow in scope and function.

3. Software Requirement Specification (SRS)

This chapter provides detailed specifications of the system under development.

3.1 Functional Requirements

Functional requirements are grouped into modules corresponding to major system components.

- Core Chatbot Module (Public Facing)
 - Multimodal Input Handling
 - * Text Input: Accept text queries in English, Urdu, and Roman Urdu (basic level).
 - * Image Input: Analyze uploaded images for contextual information (e.g., payment receipts, prescriptions, donation screenshots). Handwritten text recognition is out of scope for MVP.
 - * Voice Input (Phase 2 - Conditional): Convert voice messages to text using Speech-to-Text API. Basic Urdu support only; dialect variations are out of scope. Voice integration depends on successful testing of Phase 1 text-based system.
 - *MVP Scope: Phase 1 focuses on duo-lingual text (English + Urdu) and image input only.*
 - Intent Classification & Domain Identification
 - * Domain Classification: Automatically identify the query domain from four categories:
 - Donation: Payment methods, project details, Zakat/Sadaqah eligibility

- Healthcare: Hospital locations, services, appointments, ambulances
- Blood Services: Donor eligibility, blood bank locations, blood group availability
- General Inquiry: Organization information, volunteering, events, queries about other domains
- *Note: this does not cover the exhaustive list of data for the domains mentioned*
- * Intent Clarification Loop: When queries are ambiguous or confidence score is low, prompt user with clarifying questions before proceeding (e.g., "Are you asking about blood donation or healthcare services?").
- * Confidence Threshold: Queries with confidence score below 0.7 are flagged for human escalation.
- Contextual Response Generation
 - * RAG-Based Retrieval: Retrieve verified information from the domain-structured knowledge base using Retrieval-Augmented Generation.
 - * Persona-Aware Responses: Adapt response length, tone, and detail based on identified user persona (e.g., overseas donor, non-tech-savvy patient, recurring donor).
 - * Follow-Up Context: Maintain session context to handle follow-up questions that depend on prior conversation history.
 - * Session Management: Preserve conversation context throughout the user session to enable multi-turn dialogues.
- Multilingual Response & Accessibility
 - * Language Selection: Allow users to select their preferred language (English or Urdu) at the start of interaction.
 - * Quick Reply Options: Provide button-based shortcuts and pre-defined options for low-literacy users to minimize typing.
 - * Consistent Quality: Ensure response quality is consistent across languages with no discrepancy between English and Urdu outputs.
- Trust & Verification
 - * Branding Display: Show verified Alkhidmat branding, official logos, and chatbot verification messages to assure users of authenticity.
 - * Official Channel Verification: Display verified WhatsApp Business number and official website domain to prevent scam concerns.

- Guardrails & Safety
 - * Content Moderation: Implement guardrails to prevent inappropriate responses or off-topic conversations.
 - * Privacy Protection: Ensure no personal data is retained beyond the session lifetime.
 - * Fallback Responses: Provide predefined safe responses when the system cannot confidently answer a query.
- Information Retrieval & Data Management Module

The knowledge base contains periodically updated, domain-structured information covering (note: this is not an exhaustive list of the data for each domain):

 - RAG Knowledge Base (General Information)

Note: Knowledge base is NOT real-time; it is periodically updated by administrators.

 - * Donation Support:
 - Active and verified campaign details (healthcare, education, orphan care, women empowerment, disaster relief)
 - Payment methods and step-by-step guidance (Easypaisa, JazzCash, bank transfer, international cards, PayPal)
 - Zakat and Sadaqah eligibility criteria for different programs
 - Donation workflow and confirmation processes
 - Transparency reports and impact summaries
 - * Healthcare Information:
 - List of Alkhidmat hospitals, clinics, labs, and pharmacies with addresses and coordinates
 - Service offerings per facility (OPD, diagnostics, pharmacy, ambulance)
 - Operating hours and appointment procedures
 - Consultation fees, test charges, and medicine pricing
 - Doctor availability, specializations, and gender-specific staff information
 - Emergency helpline numbers and ambulance booking procedures
 - * Program Information:
 - Summaries of Alkhidmat’s 12+ domains (education, water, orphan care, community services, disaster management)

- Women-centric initiatives (Bano Qabil, maternity services, women empowerment)
- Youth and student programs
- Corporate Social Responsibility (CSR) partnership opportunities
- * General Information:
 - Organization history, mission, and credibility verification
 - Volunteering opportunities and community event schedules
 - FAQs on trust, transparency, and fund utilization
- Blood Donation Microservice (Operational Data)

A separate microservice containing transactional and inventory information integrates with the main system to handle blood donation queries:

 - * Donor Eligibility Validation: Check eligibility based on age, weight, hemoglobin levels, blood pressure, and donation intervals (WHO and Alkhidmat internal SOPs).
 - * Blood Bank Locations: Retrieve nearest Alkhidmat-licensed blood bank addresses and contact information based on user location.
 - * Blood Group Availability: Query current blood stock levels and urgent blood group requirements.
 - * Donation Process Information: Provide step-by-step donation process details (registration, screening, donation, post-care instructions).
 - * Donor Network Registration: Allow users to register for SMS/WhatsApp alerts when their blood group is urgently needed.
 - * Post-Donation Guidelines: Provide medical instructions (hydration, rest, when to resume activities).
 - * Recipient Support: Guide blood recipients on required documents (CNIC, prescription), cross-match duration, reservation policies, and service charges.
- User Profiling & Persona Management (Contextual Database)

Maintain a separate database to store user information and enable personalized interactions:

 - * Persona Classification: Classify users into personas based on interaction patterns using a donor-beneficiary matrix:
 - Donor personas: Overseas, Local, Recurring, First-Time, Corporate, Non-tech Savvy

- Healthcare personas: Rural/Urban Patient, New Patient, Non-tech Savvy Patient
 - Blood service personas: New/Recurring Blood Donor, Blood Recipient
- * Interaction History Tracking: Record user query patterns, preferred language, interaction frequency, and topics of interest.
- * Dynamic Persona Updates: Update user persona classification after each interaction based on new behavioural data.
- * Context-Aware Response Adaptation: Tailor response length, explanation depth, and recommendations based on identified persona (e.g., provide detailed explanations for first-time donors, concise responses for recurring donors).
- * Personalized Recommendations: Suggest relevant campaigns, services, or programs based on user history and persona.
- Human Agent Dashboard (Staff Facing)
 - Escalation & Routing
 - * Automatic Escalation: Forward queries with confidence score below 0.7 to relevant department-based chat agents.
 - * Domain-Based Routing: Route escalated queries to specialized agents (Donation team, Healthcare team, Blood Services team, General Support).
 - * Sensitive Query Handling: Automatically escalate queries involving complaints, refunds, or personal grievances to human agents.
 - Conversation Management
 - * Real-Time Takeover: Allow human agents to join ongoing conversations within the same chat thread without disrupting user experience.
 - * 24-Hour WhatsApp Window: Enable agents to respond within the 24-hour WhatsApp Business API interaction window.
 - * Pre-Approved Templates: Provide agents with Meta-approved message templates for initiating conversations outside the 24-hour window.
 - * Conversation History Access: Enable agents to view full conversation context before responding.
 - Knowledge Base & Dialogue Management

- * Content Updates: Allow administrators to add, modify, or archive FAQs and program information in the RAG knowledge base.
- * Dialogue Flow Definition: Enable admins to define and update pre-approved greeting messages, fallback responses, and guided conversation flows.
- * Guardrail Configuration: Provide admin interface to set and adjust content moderation rules and response boundaries.
- Role-Based Access Control
 - * Chat Agent Access: Grant domain-specific agents access only to queries within their department.
 - * Admin Privileges: Provide full system access to administrators for knowledge base management, guardrail settings, and user management.
 - * Audit Trail: Log all agent actions (view, respond, resolve) with timestamps for accountability.
- Analytics & Logging Module
 - Usage Tracking
 - * Query Metrics: Record total queries, query volume by domain, peak usage times, and average session duration.
 - * Response Time Monitoring: Track response time per query (target: 3-15 seconds based on complexity).
 - * Escalation Rate Tracking: Monitor percentage of queries escalated to human agents by domain.
 - User Feedback Collection
 - * Post-Interaction Rating: Prompt users for thumbs-up/thumbs-down rating 15-20 minutes after their last query.
 - * Follow-Up Query Check: Send a message asking if the user has any remaining questions before requesting feedback.
 - * Satisfaction Score Calculation: Aggregate user satisfaction metrics over time.
 - Performance Analysis
 - * Answer Accuracy Monitoring: Track percentage of queries resolved by AI without escalation (target: $\geq 90\%$).

- * Domain Performance: Analyze which domains have highest accuracy, longest response times, or most escalations.
 - * Trend Identification: Highlight common query topics and identify emerging information needs that require knowledge base updates.
- Reporting & Insights
 - * Dashboard Visualization: Provide real-time analytics dashboard for administrators showing KPIs and trends.
 - * Export Functionality: Generate CSV/PDF reports of usage metrics for management review.
 - * Anomaly Detection: Flag unusual query patterns (e.g., spam, repeated failures, sudden traffic spikes).
- Out of Scope
 - Handling of regional Urdu dialects is beyond the current scope. The system is designed to support only standard Urdu syntax and semantics.
 - The chatbot does not process handwritten text to extract text from images.
 - Support for additional regional languages such as Punjabi, Sindhi, Pashto, or Balochi is out of scope. The MVP exclusively supports Urdu and English text input.
 - The chatbot does not produce voice-based or audio-generated replies. All communication remains text-based for now.
 - The LLM does not store personal data beyond basic identifiers (name, contact number, email). If such data already exists in the Alkhidmat database, it is retrieved for continuity; otherwise, the LLM treats the user as new. No long-term memory or profiling is maintained.
 - Real-time data access is out of scope. The LLM retrieves information from the existing database, which requires manual updates. If the database is not updated, the system will rely on previously stored information and cannot refresh data automatically.
 - Following features are out of scope as for phase 1 of development; In case of a successful phase 1 completion, these will be added to the scope.
 - * Bilingual speech input support for chatbot interaction, enabling automatic transcription and intent detection from spoken English or Urdu.

- * Enhanced understanding and generation of Roman Urdu text for more natural and localized user communication.
- * Advanced WhatsApp broadcasting and personalized campaign messaging to improve outreach effectiveness.

3.2 Use Cases

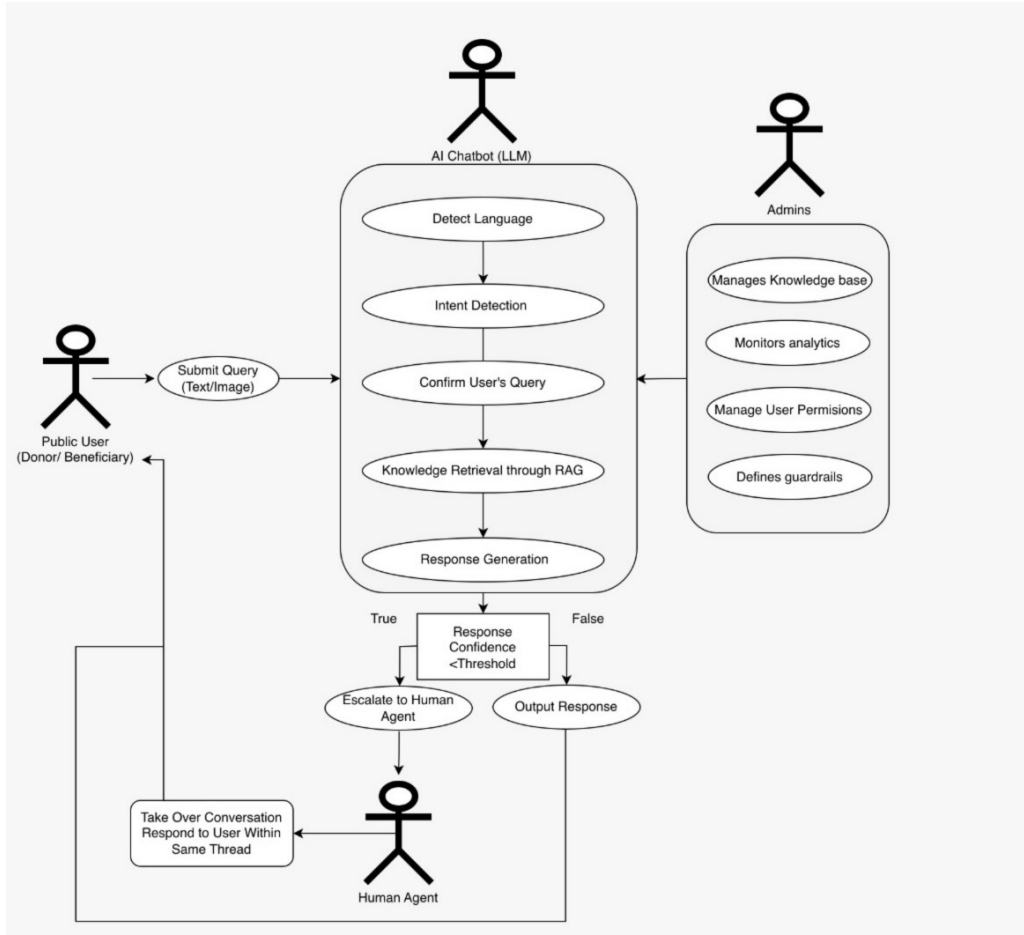


Figure 3.1: Use Case Diagram of the Public Chat Portal for Alkhidmat

3.3 Non-functional Requirements

Table 3.1: Non-Functional Requirements

Category	Description	Example
Performance	The system should respond within 3 to 15 seconds per query, depending on the complexity.	Simple queries (e.g., blood bank locations) should return results in 3 seconds. Complex queries (e.g., eligibility criteria) may take up to 15 seconds.
Accuracy and Relevance	Ensure the system provides $\geq 90\%$ accuracy and relevance in its responses to ensure reliability.	If a user asks for blood donation instructions, the response should be factually correct and relevant in 90% of cases.
Usability / Accessibility	Support multilingual responses with consistent quality in both languages (e.g., English and Urdu). Include clear button-based flows for non-technical users.	The system should provide responses in Urdu with the same clarity and accuracy as in English, without discrepancies.
Reliability	Ensure the system maintains $> 90\%$ answer accuracy and minimal human routing. If accuracy falls below a threshold, route to a human agent.	If a user asks for donation details and the system's confidence is below 70%, it should route to a human agent.
User Satisfaction (KPI)	Measure user satisfaction by asking for feedback 15–20 minutes after the last query with thumbs up/down ratings.	After a session, if the user does not respond or replies with “no,” send a rating question (thumbs up/down) to evaluate the system's effectiveness.
Data Integrity & Backup	Ensure data accuracy and periodic backups of important data (e.g., RAG knowledge base). Implement automated weekly backups and data validation.	Perform weekly automated backups of key data and ensure that data validation occurs during data entry to prevent corruption.

Continued on next page

Table 3.1 (continued): Non-Functional Requirements

Category	Description	Example
Security / Compliance	Implement end-to-end encryption for all communications. Ensure no personal data retention beyond session lifetime.	Personal data should not be retained after the session ends. The system should use encrypted communication protocols to secure sensitive information.
Scalability	Support ≥ 500 simultaneous users without performance degradation using FastAPI + Elasticsearch on Docker/Heroku.	The system should scale to support 500 simultaneous users accessing the platform without any degradation in response time.
Maintainability	Use modular code architecture with clear documentation and API version control to ensure the system is easy to maintain and update.	The codebase should be modular with proper documentation to allow for easy updates and troubleshooting.
Compliance and Ethics	Adhere to Pakistan's data privacy guidelines and AI ethics principles for NGO use.	Ensure that the system follows local data privacy regulations and complies with ethical AI practices for non-profit organizations.

3.4 System Diagram

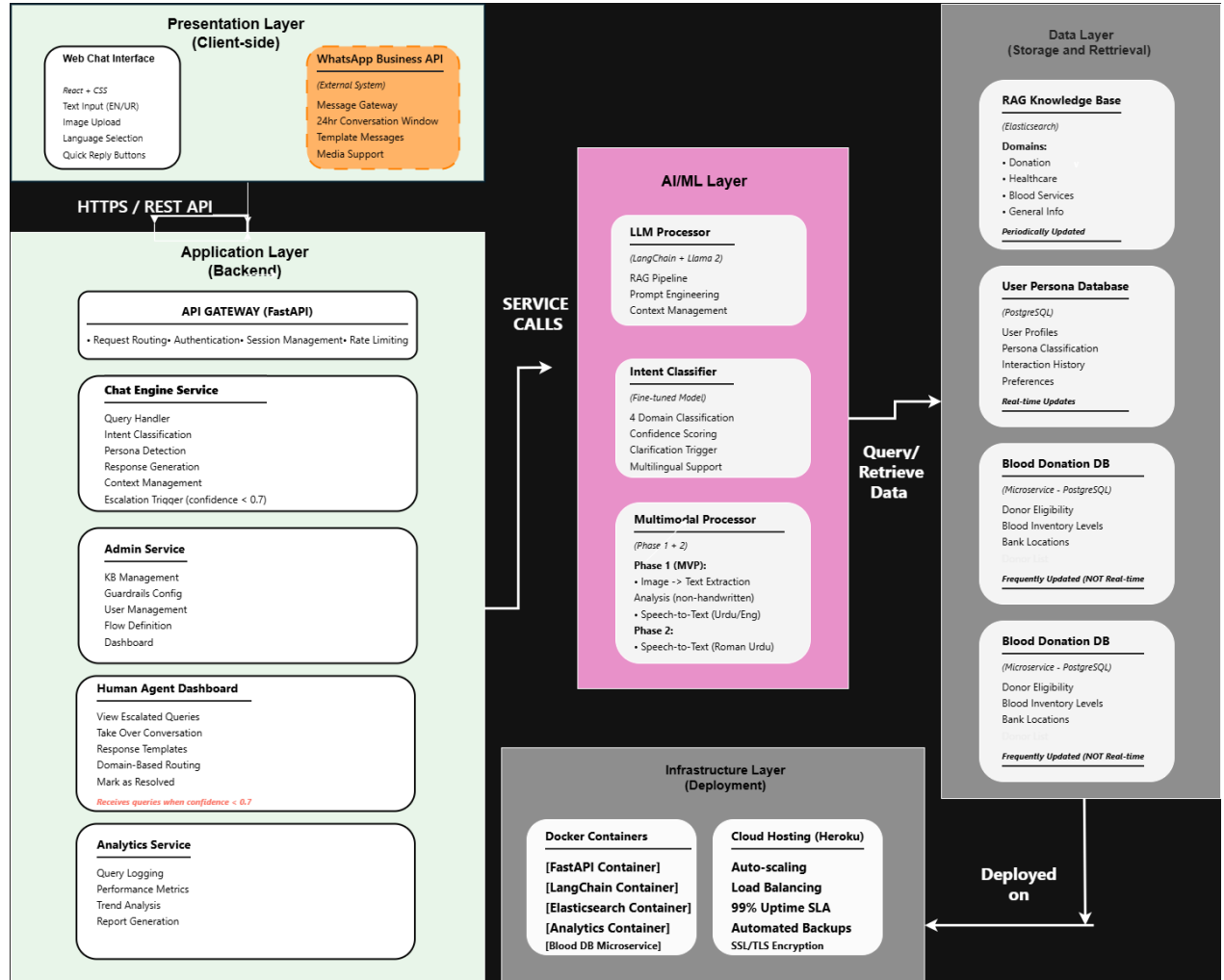


Figure 3.2: High-level System Architecture

To view the diagram clearly, kindly access the following link.

The proposed system follows a five-layer architecture: Presentation Layer, Application Layer, AI/ML Layer, Data Layer, and Infrastructure Layer. Each layer plays a distinct role in ensuring efficient communication, scalability, and performance.

3.4.1 Presentation Layer (Client-Side)

Components:

- **Web Chat Interface:** Built using React and CSS, enabling the user to interact with the system through a web interface. It supports features like text input (EN/UR), image uploads, language selection, and quick reply buttons.
- **WhatsApp Business API:** Used for external messaging functionalities such as managing conversations within a 24-hour window, sending template messages, and supporting media in chats.

Role: This layer is responsible for presenting the interface to the user, collecting user inputs, and initiating requests to the application layer via HTTP/REST APIs.

3.4.2 Application Layer (Backend)

Components:

- **API Gateway:** Manages incoming requests, handles authentication, session management, and rate limiting.
- **Chat Engine Service:** Handles the main logic for query processing, intent classification, persona detection, and response generation. It also includes context management and escalation triggers for low-confidence queries.
- **Admin Service:** Provides tools for managing the knowledge base, user roles, and defining conversation flows.
- **Human Agent Dashboard:** Assists human agents in handling escalated queries, managing conversations, and routing them based on specific domains.
- **Analytics Service:** Logs queries, monitors performance metrics, and generates reports on system usage and trends.

Role: This layer is where the core processing happens. It handles requests from the presentation layer, processes them using the chat engine, and provides feedback to the user.

3.4.3 AI/ML Layer

Components:

- **LLM Processor:** Processes the natural language input using large language models (LLMs) such as LangChain and Llama 2. It performs tasks like prompt engineering and context management.
- **Intent Classifier:** Classifies user inputs into predefined domains, scores confidence levels, and triggers escalation when confidence is low.
- **Multimodal Processor:** Handles input types like images or non-handwritten text and processes them into usable information, including support for Urdu and Roman Urdu.

Role: The AI/ML layer enhances the system's ability to process and understand user queries, making it capable of handling diverse input types and languages while maintaining context for more accurate responses.

3.4.4 Data Layer (Storage and Retrieval)

Components:

- **RAG Knowledge Base:** Stores structured data related to donation, health-care, and general services. This database is periodically updated.
- **User Persona Database:** Tracks user profiles, including interaction history, preferences, and persona classification, to provide personalized responses.

Role: The data layer ensures that the necessary information is available for the application layer to process and retrieve during interactions. It stores both real-time and frequently updated data for accurate responses.

3.4.5 Infrastructure Layer (Deployment)

Components:

- **Docker Containers:** Each component, including the FastAPI service, LangChain, Elasticsearch, and analytics, is containerized to ensure consistent and scalable deployment.

- **Cloud Hosting (Heroku):** The system is hosted on Heroku with auto-scaling, load balancing, and high availability (99% uptime SLA). SSL/TLS encryption ensures secure communication.

Role: The infrastructure layer ensures that the system is deployed efficiently, scalable, and resilient. It handles the deployment of services and ensures they are readily available to process requests.

3.4.6 Relationships Between Layers

- User interaction starts at the **Presentation Layer**, where the user inputs queries.
- Requests are routed via HTTPS/REST APIs to the **Application Layer** for processing.
- The **Chat Engine Service** in the Application Layer utilizes the **AI/ML Layer** to classify intents and process natural language queries.
- **Data Retrieval** from the Data Layer occurs through queries made by application services, which fetch relevant data from the knowledge base and databases.
- The system is deployed on the **Infrastructure Layer**, ensuring scalability and security.

3.4.7 Flow of Interaction

1. **User Interaction** → Presentation Layer → API Gateway
2. **Request Routing** → Chat Engine Service → AI/ML Layer (Intent Classification & Response Generation)
3. **Data Retrieval** → Data Layer (RAG Knowledge Base, User Persona)
4. **Response Sent** → Presentation Layer → User (via WhatsApp or Web Chat)

This architecture ensures that each layer is responsible for a specific aspect of the system, from front-end interaction to backend processing, AI-based intent classification, and data management, all underpinned by scalable and secure infrastructure.

3.5 External Interfaces

3.5.1 User Interfaces - Wireframes

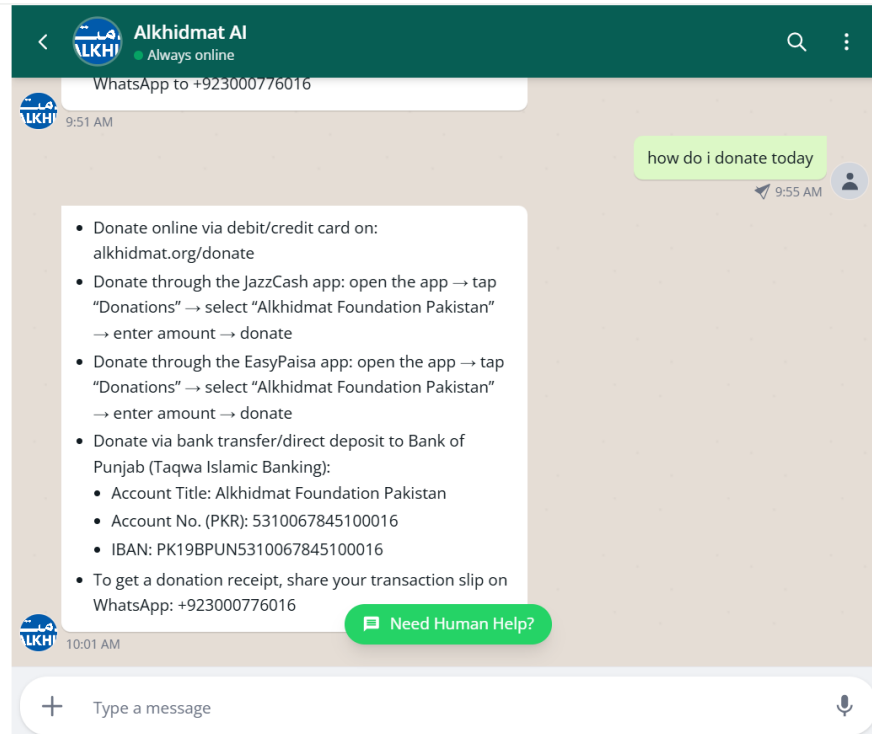


Figure 3.3: User Interface using Whatsapp

The two wireframes show how users might interact with the AI chatbot. Here, two use-cases are shown; monetary donors can select between Zakat, Sadaqah, or General Donations using quick buttons, receive verified payment options (Easypaisa, JazzCash, Visa/MasterCard), and obtain secure links with automated receipts, while complex or high-value cases are routed to human donation officers. Similarly, users seeking blood support can provide their blood group and city to instantly retrieve nearby Alkhidmat blood bank details and contact numbers; for urgent or unclear cases, the chatbot escalates the conversation to a live human agent within the same WhatsApp thread as we see in below wireframes. Both flows ensure verified information, user-friendly navigation, and human oversight for transparency and reliability.

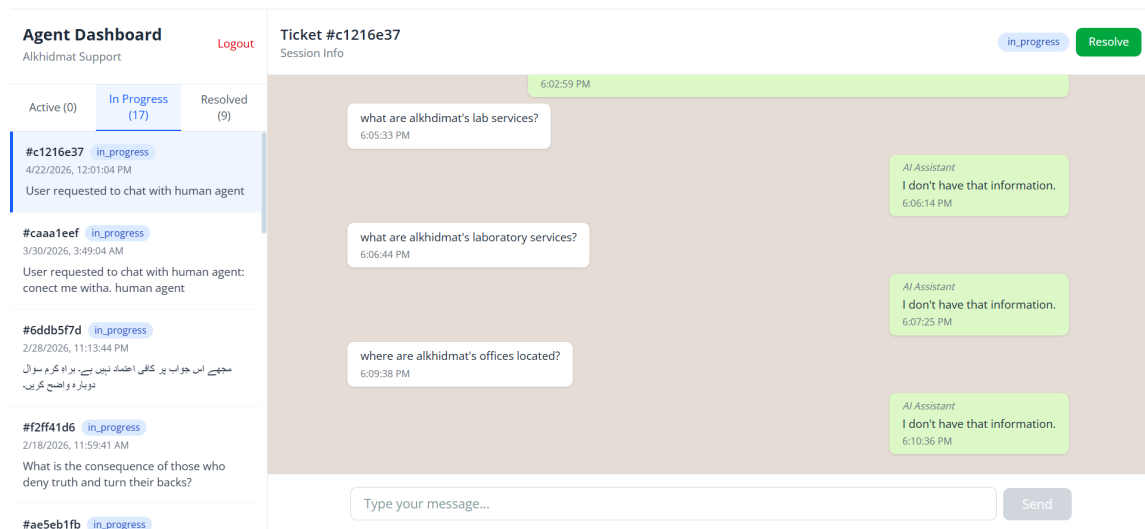


Figure 3.4: Staff Dashboard

If the AI bot is unable to confidently resolve any query, it will be escalated to Al-Khidmat's staff. The following is the dashboard where they will receive the queries. The following example shows that the AI bit resolved a query with 8% confidence, so it has been escalated to Fatima Ahmed, a human agent. She sees that there is a new ticket. She reviews the context where the user asked sensitive information and would personally guide her - all while updating the knowledge base for future training. The dashboard includes:

- Left Panel: Live ticket queue with 4 realistic escalation scenarios (medical emergency, zakat calculation, scholarship, etc)
- Center Panel: Bilingual conversation thread showing bot-to-human handoff, where the agent has an option to transfer the query to another agent and resolve it once completed.
- Right Panel: User profile, escalation metadata, and knowledge base update form

Agent Dashboard

Alkhidmat Support

Logout

Active (0)

In Progress (17)

Resolved (9)

#c1216e37 in_progress

4/22/2026, 12:01:04 PM

User requested to chat with human agent

#caaa1eef in_progress

3/30/2026, 3:49:04 AM

User requested to chat with human agent:
conect me witha. human agent

#6ddb5f7d in_progress

2/28/2026, 11:13:44 PM

مجھے اس جواب پر کافی اعتماد نہیں ہے۔ براہ کرم سوال دوبارہ واضح کریں۔

#f2ff41d6 in_progress

2/18/2026, 11:59:41 AM

What is the consequence of those who deny truth and turn their backs?

Figure 3.5: Further Contents in Ticket Content Panel

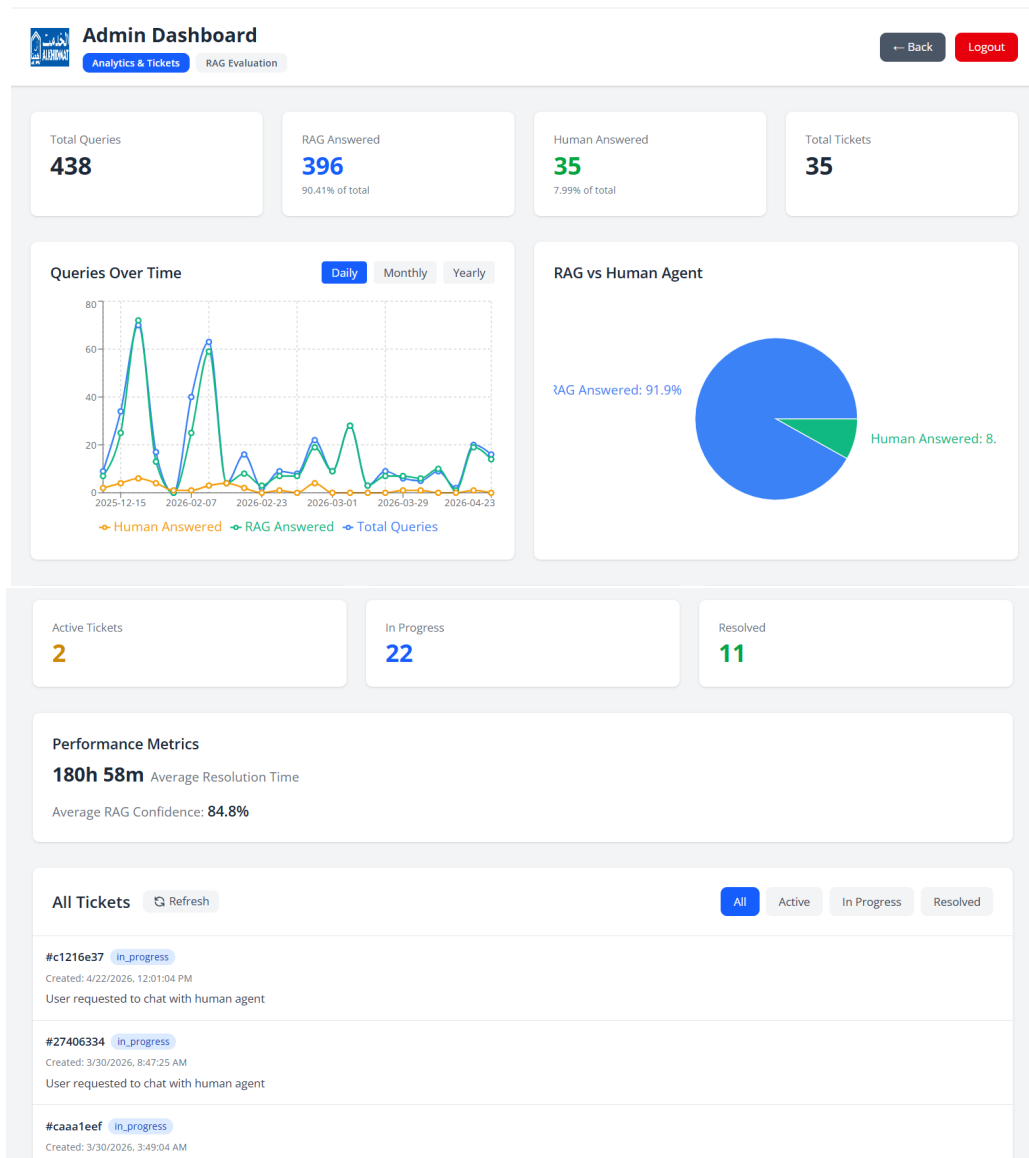


Figure 3.6: Admin’s Dashboard - Analytics Monitor

The Analytics Dashboard features real-time tracking of chatbot performance and user activity, including domain-wise query distribution, daily and hourly interaction trends, and key usage metrics like total queries, average session time, and peak hours. It provides detailed insights into escalation rates, comparing AI-handled and human-assisted queries across domains to assess accuracy and efficiency. Additional

features include a date selector for reviewing past analytics, visual charts for quick interpretation, and an intuitive, admin-friendly interface aligned with Alkhidmat Foundation’s branding for easy monitoring and performance optimization.

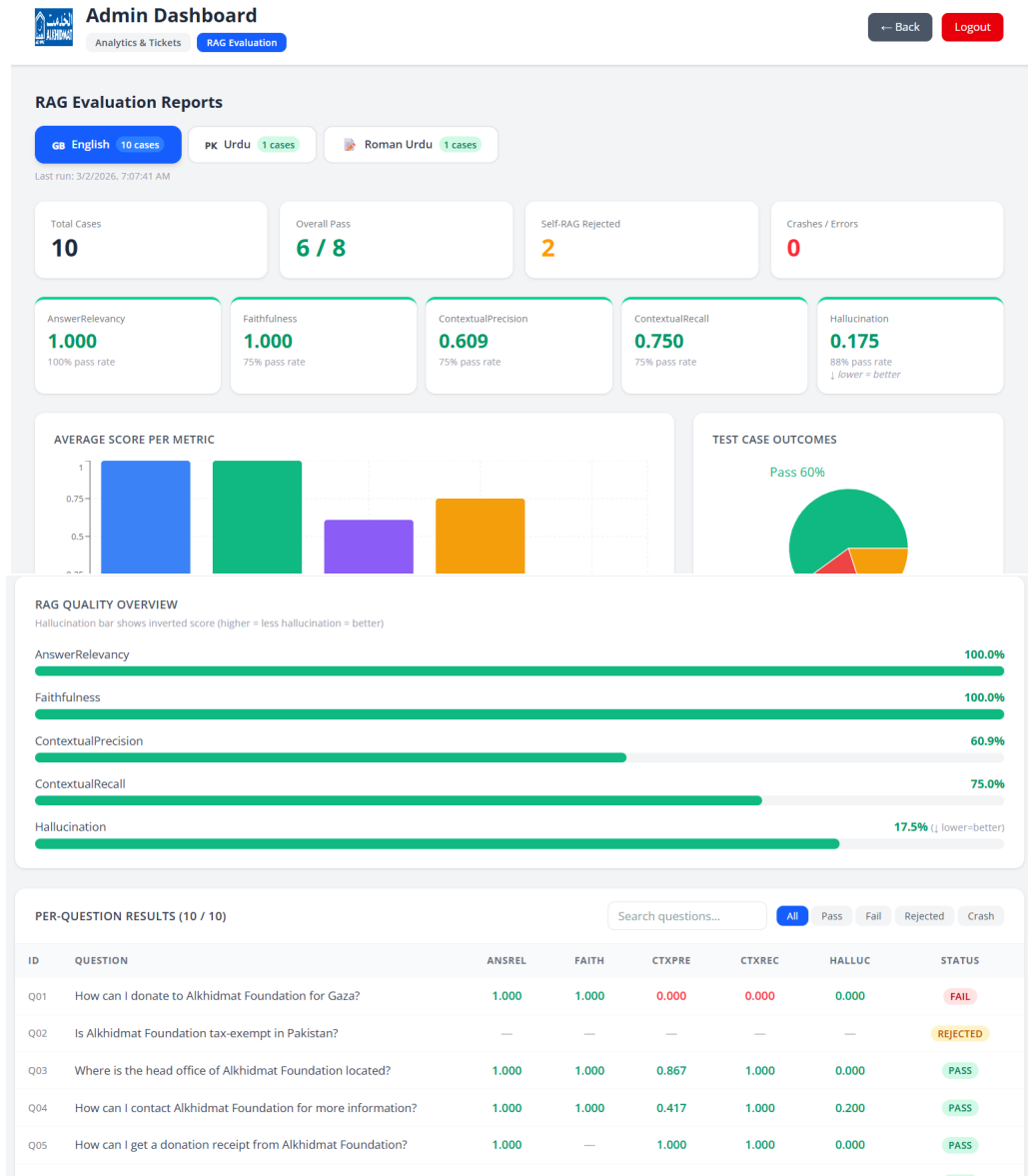


Figure 3.7: Admin’s Dashboard - RAG Evaluation

4. Software Design Specification (SDS)

4.1 Software Design

4.1.1 High-level Architecture

Five-layer architecture (as in SRS) 3.2. This high-level system architecture shows the five-layer structure and component interactions.

Flow (short): Client → API Gateway → Chat Engine → (Intent Classifier → RAG Retrieval + LLM) → Response → Presentation. Escalation path sends conversation + context to Human Agent Dashboard; Human updates KB via Admin Service.

4.1.2 UML Class Diagram (PlantUML)

The system components and their relationships are shown in the following UML class diagram:

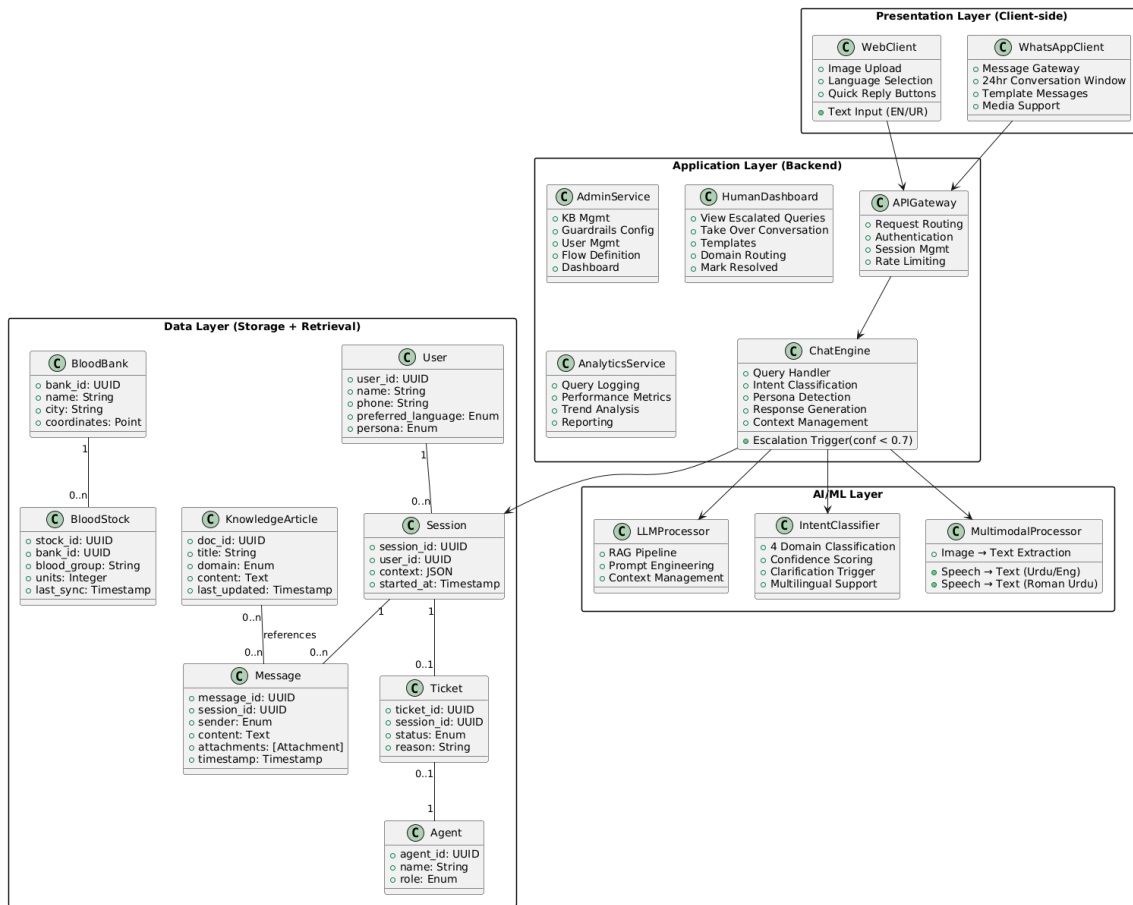


Figure 4.1: UML Class Diagram showing the main system components, their relationships, and data entities

4.1.3 Classes and Services

Application-Layer Services

APIGateway

- **Purpose:** Authentication, rate-limiting, routing to backend services, request validation.
- **Attributes:** tokenStore (JWT secret), rateLimitConfig.
- **Methods:** authenticate(req), authorize(req), route(req), rateLimit(req).

- **Relationships:** Front door for WebClient & WhatsAppClient → ChatEngine, AdminService.

ChatEngine

- **Purpose:** Orchestrates conversational processing: receives message, manages session, calls intent classifier, multimodal processor, RAG engine, and returns responses.
- **Attributes:** sessionStore (Redis), kbClient, classifierClient, ragClient, escalationPolicy.
- **Methods:**
 - processIncoming(message: Message) – main entry.
 - classifyIntent(text) → IntentResult (intent, confidence).
 - handleMultimodal(attachments) → structured artifacts.
 - generateResponse(context) → final message (via RAGEngine + prompt).
 - escalate(session, reason) → create Ticket and route to HumanDashboard.
- **Relationships:** calls IntentClassifier, MultimodalProcessor, RAGEngine, DataStore.

AdminService

- **Purpose:** CRUD for knowledge base, guardrails, role management.
- **Attributes:** KBRepository, userManager.
- **Methods:** createArticle(), updateArticle(), publishKBVersion(), setGuardrails().
- **Relationships:** updates KnowledgeArticle entries used by RAGEngine.

HumanDashboard (HumanAgent)

- **Purpose:** Staff-facing UI + API for taking over escalated tickets, replying to users, updating KB entries.
- **Attributes:** ticketQueue, websocketConnections.
- **Methods:** claimTicket(ticketId), postReply(ticketId, message), updateKBFromInteraction(...).
- **Relationships:** Ticket ↔ Agent ↔ Session.

AnalyticsService

- **Purpose:** Log metrics, build KPIs (accuracy, latency, escalation rates).
- **Attributes:** metricsStore, logDB.
- **Methods:** logQuery(...), getKPI(start,end), alertAnomalies().
- **Relationships:** receives events from ChatEngine and AdminService.

AI/ML classes/services

IntentClassifier

- **Purpose:** Predicts domain & intent from textual input; returns confidence score.
- **Attributes:** model (transformer or lightweight embed+classify), labelMap.
- **Methods:** predict(text) $\rightarrow$ {intent, domain, confidence, top_k}.

MultimodalProcessor

- **Purpose:** Process images (receipt, prescription), run OCR, detect entities, extract text & metadata.
- **Attributes:** visionModel, ocrEngine (Tesseract or cloud), imagePreproc.
- **Methods:** processImage(img) $\rightarrow$ {text, tags, doc_type, confidence}.

RAGEngine

- **Purpose:** Retrieval-Augmented Generation: vector search $\rightarrow$ rerank $\rightarrow$ LLM prompt $\rightarrow$ answer.
- **Attributes:** vectorStore, index (Elasticsearch / FAISS), llmClient, prompt-Templates.
- **Methods:** retrieve(query,k), buildPrompt(context, docs), generateAnswer(prompt) $\rightarrow$ answer, sources.

Data-layer domain models (DB entities)

User

- **Attributes:** user_id (PK), name, phone, email, preferred_language, created_at.
- **Methods (service-level):** getProfile(), updatePreferences().

Session

- **Attributes:** session_id (PK), user_id (FK), started_at, last_active_at, context (JSON).
- **Methods:** loadContext(), saveContext().

Message

- **Attributes:** message_id (PK), session_id (FK), sender (enum), content, attachments (JSON), timestamp, intent_label, confidence.
- **Methods:** serialize().

Ticket

- **Attributes:** ticket_id (PK), session_id (FK), status (Open/InProgress/Resolved), assigned_to (agent_id FK), priority, reason, created_at, resolved_at.
- **Methods:** assign(agent), resolve().

KnowledgeArticle

- **Attributes:** doc_id (PK), title, content, domain, tags, source_url, last_updated, version.
- **Methods:** searchIndex(), getExcerpt().

Agent

- **Attributes:** agent_id (PK), name, roles, email, active.
- **Methods:** claimTicket().

4.2 Data Design

This section presents the normalized relational model, cardinalities, and the core relational schema. The model is designed to conform to third normal form (3NF).

4.2.1 Entities, Attributes, Primary/Foreign Keys, and Cardinalities

The primary relationships between entities are summarized as follows:

- **User (1) – (0..n) Session:** One user can have many sessions. This design allows tracking the number of interactions and building user-based context for future queries.
- **Session (1) – (0..n) Message:** A session contains many messages.
- **Session (1) – (0..1) Ticket:** A session may create one open ticket.
- **Ticket (0..n) – (0..1) Agent:** A ticket may be assigned to one agent; agents handle many tickets.
- **KnowledgeArticle (0..n) – (0..n) Message (reference):** Messages (bot replies) may reference multiple knowledge articles and vice versa, implemented through a junction table.

4.2.2 Relational Schema (Core Tables)

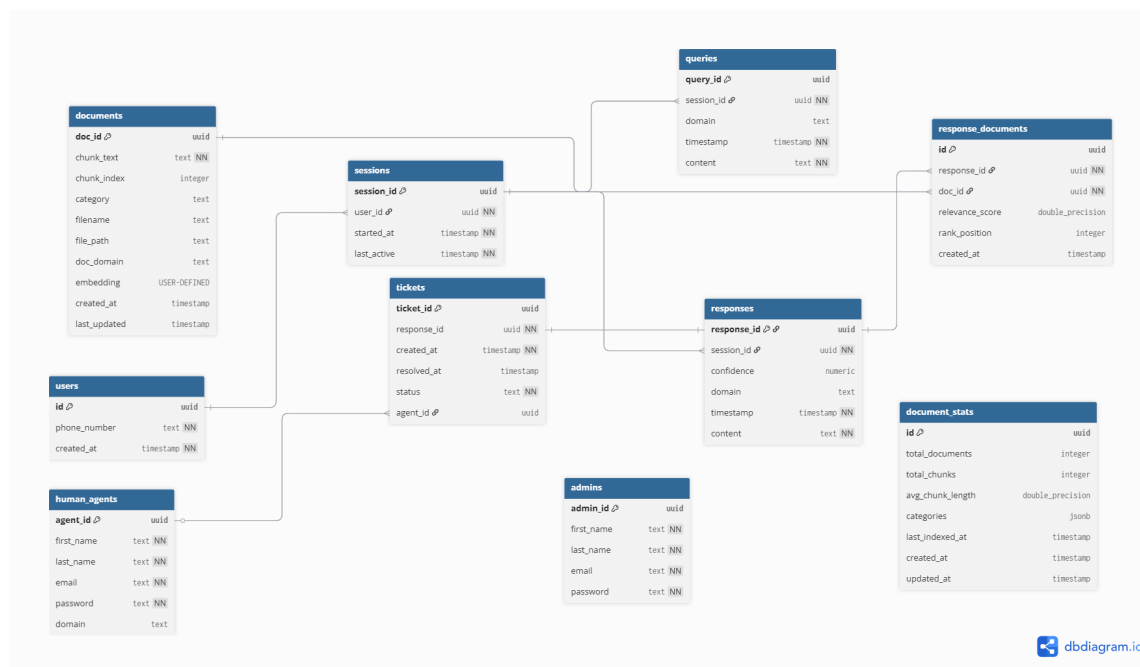


Figure 4.2: Relational schema diagram showing core tables, attributes, primary keys (underlined), and foreign key relationships

4.2.3 Indexes and Search

The indexing strategy includes the following components:

- Full-text indexing using either Elasticsearch or PostgreSQL tsvector on the `knowledge_articles.content` and `knowledge_articles.title` fields.
- Vector embeddings are stored in a dedicated vector store (FAISS, Milvus, or Elasticsearch vector index), keyed by `doc_id`. Vector metadata is maintained in a lightweight `kb_index` table that establishes the mapping between `doc_id` and `vector_id`.

4.2.4 Normalization and Cardinalities Rationale

The schema design incorporates the following principles:

- The schema explicitly separates knowledge articles (the canonical knowledge base) from message-level references. This separation avoids data duplication while supporting many-to-many referencing relationships, maintaining third normal form (3NF).
- The `sessions` table stores transient conversation context, while messages are treated as transactional records that cascade upon session deletion.
- The `message_article_refs` junction table preserves provenance information, explicitly recording which knowledge base articles supported a specific bot response.

4.2.5 Technical Details

Data Pipeline

The system implements an end-to-end data pipeline for processing both unstructured and structured data sources:

1. **Web Scraping:** Periodic crawlers extract content from designated websites, parse structured fields, and store raw output with checksums in a staging area.
2. **Document Repository:** Verified content is persisted in Google Drive with comprehensive metadata including domain classification, source URL, and checksums.
3. **Preprocessing:** Text cleaning, language detection, Roman-Urdu normalization, semantic-aware chunking (300–500 tokens), and image OCR processing.
4. **Embedding and Indexing:** Document chunks and structured data summaries are converted to embeddings and stored in a vector database with associated metadata.
5. **RAG Retrieval and Generation:** Query embedding followed by vector search, result reranking, prompt construction with retrieved context, and LLM generation.

Retrieval Strategy

The retrieval process combines vector similarity search with metadata filtering and reranking:

- Initial retrieval of top-k candidate documents using vector similarity
- Reranking based on multiple signals: cosine similarity, document recency, domain relevance, and exact match criteria (city, blood group)
- Selection of top-n documents for prompt construction

Prompt Construction and Generation

Responses are generated using carefully structured prompts that explicitly constrain the LLM to use only provided source documents:

SYSTEM: You are a domain-expert assistant. Use only the documents listed in SOURCES.

SOURCES: [numbered document excerpts with identifiers]

TASK: Provide a concise, evidence-based answer. For each factual claim include the relevant source identifier. If no verified information is available in the provided sources, explicitly state "No verified information is available."

Image Processing Pipeline

All uploaded and scraped images undergo comprehensive processing:

- Preprocessing including resizing, denoising, deskewing, and grayscale conversion
- OCR processing with language hinting for English, Urdu, and Roman-Urdu
- Post-OCR named entity recognition and structured extraction for key entities
- Confidence-based quality control with fallback handling for low-confidence results

Language Handling

The system addresses multilingual requirements through:

- Language detection for input classification
- Roman-Urdu normalization using mapping tables and transliteration heuristics
- Multilingual embedding models capable of representing semantic similarity across English, Urdu, and Roman-Urdu content

Monitoring and Quality Control

The system includes comprehensive monitoring and validation mechanisms:

- Post-generation source coverage verification to ensure response content is supported by retrieved documents
- Confidence scoring based on retrieval quality, source coverage, and supporting evidence
- Escalation mechanisms for low-confidence responses and explicit user requests for human intervention
- Metrics collection including query latency, knowledge base hit rate, escalation rate, and retrieval precision

Failure Modes and Mitigations

- **Stale stock information:** Explicit timestamp presentation and recommendation that recipients verify availability by contacting blood banks directly
- **OCR processing failures:** Confidence-based detection with user prompts for image reupload
- **Retrieval failures:** Fallback to keyword search or predefined FAQ responses
- **Hallucination prevention:** Strict prompt grounding, source verification, and low-temperature generation parameters

4.2.6 Summary

The described system architecture implements a comprehensive Retrieval-Augmented Generation framework specifically tailored for blood donation and healthcare support applications. The architecture combines structured operational data management with extensive knowledge base indexing, sophisticated retrieval and reranking mechanisms, and robust multimodal processing capabilities. The design emphasizes factual grounding through explicit source attribution, confidence-based quality control, and comprehensive escalation pathways to ensure reliable, verifiable responses across all operational scenarios.

5. Methodology, Experiments and Results

5.1 System Architecture: Agentic RAG and Frontend Integration

The system is a full-stack application built across three layers: a React-based frontend, a FastAPI backend, and an Agentic Retrieval-Augmented Generation (RAG) reasoning engine. Standard RAG systems retrieve documents and pass them directly to a language model for generation, with no mechanism to verify whether the retrieved context is sufficient or whether the generated response is grounded in it. This architecture extends that baseline by wrapping the retrieval-generation pipeline in a layer of autonomous agents that regulate what gets retrieved, validate what gets generated, and escalate when confidence is insufficient. The result is a system designed to serve a multilingual, low-digital-literacy user base while maintaining factual reliability.

5.1.1 Frontend Architecture and User Interface

The frontend is built using **React.js 18**, **Vite**, and **Tailwind CSS**. React.js 18 was selected for its component-based architecture, which allows role-specific views (user, agent, admin) to be developed and maintained independently without duplicating shared logic. Vite replaces the traditional Create React App bundler, offering significantly faster hot module replacement during development and smaller production bundles, important for a deployment environment where backend response times are already high. Tailwind CSS enables rapid, consistent styling without a custom stylesheet layer.

The interface is structured around three role-based dashboards:

- **User Interface:** Accessible via OTP-based phone authentication — chosen over password login because the target demographic (NGO service recipients) may not maintain email accounts or remember passwords. The interface supports English, Urdu (Nastaliq script), and Roman Urdu, with language switching at runtime. This is not cosmetic: the backend routes queries, selects retrieval strategies, and selects generation models based on the detected language, so the frontend language state is directly coupled to backend behaviour.
- **Human Agent Dashboard:** Displays escalated tickets, supports real-time chat with users via WebSocket, and allows agents to assign and resolve tickets. WebSocket was used over polling because escalated sessions require low-latency, bidirectional communication and polling at acceptable intervals would generate unnecessary server load given the system’s existing processing overhead.
- **Admin Dashboard:** Provides analytics on query volume, domain distribution, RAG versus human agent resolution rates, and ticket resolution times. Along with this, there is a detailed Deepeval analysis of the RAG system’s performance. This data serves a monitoring function: it allows administrators to identify domains where the RAG system underperforms and where the knowledge base requires updates. Moreover, admins can oversee all the tickets, as well as improve the RAG system by adding new documents to the knowledge base, and adjusting prompts and retrieval strategies based on observed performance.

The system also accepts image and PDF uploads. These are processed through an OCR pipeline combining **Tesseract** and **PaddleOCR**. Tesseract handles standard printed text, while PaddleOCR provides stronger performance on structured layouts such as medical receipts and donation forms, where tabular content would be misread by a single-engine pipeline. Extracted text is appended to the user query as grounding context before retrieval.

5.1.2 Agentic RAG Reasoning Engine

The core limitation of a vanilla RAG system is that it retrieves and generates unconditionally: every query triggers embedding, every retrieval result is passed to the model, and every generated response is returned to the user regardless of its factual grounding. In an NGO context — where users ask about healthcare eligibility, donation procedures, and beneficiary rights — an ungrounded response is not merely unhelpful; it can cause direct harm. The agentic layer addresses this by introducing four components that each enforce a specific constraint.

Router Agent

Before any embedding is computed, the Router Agent classifies whether the query requires retrieval at all. Queries such as greetings, expressions of gratitude, or clearly out-of-domain questions (e.g., “how is the weather today”) carry no informational intent that the knowledge base can satisfy. Without this check, the system would generate embeddings, execute a vector search, and pass retrieved documents to the LLM for every such query; wasting computation and, more critically, risking hallucination.

The risk of skipping this step was observed empirically during baseline testing. When the Router Agent was not active, a query of “how is the weather today” caused the base LLM to acknowledge it lacked real-time data, then proceed to generate unsolicited, ungrounded claims about climate change and flooding in Pakistan. None of this content existed in the knowledge base. With the Router Agent active, the same query was classified as out-of-domain and rejected with a direct, accurate response: “That is an irrelevant question.” The model was prevented from generating at all.

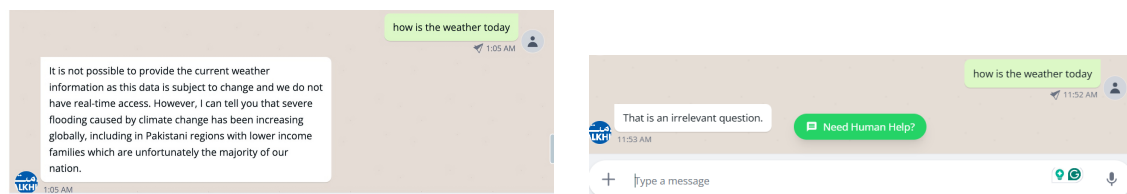


Figure 5.1: Hallucination comparison on an out-of-domain query. *Left*: Baseline LLM without the agentic layer generates an unsolicited, ungrounded response connecting weather to climate change in Pakistan. *Right*: Full pipeline with Router Agent active correctly classifies the query as out-of-domain and declines to generate.

By intercepting non-informational queries before retrieval, the Router Agent reduces unnecessary embedding computation and eliminates a significant class of hallucination at the source.

Retriever Agent with Fallback

For queries that require retrieval, the Retriever Agent generates embeddings using **intfloat/multilingual-e5-base** — a 768-dimensional multilingual model chosen because the knowledge base and user queries span English, Urdu, and Roman Urdu. A monolingual English model would produce poor cross-lingual similarity scores; multilingual-e5-base is trained to align semantically equivalent content across languages in a shared vector space.

Retrieval applies a similarity threshold of ≥ 0.6 . If fewer than two documents exceed this threshold, the system does not proceed with an under-supported context window. Instead, a fallback retrieval pass is triggered using a broader general-domain query to expand coverage. This threshold was set to balance precision against recall: a higher threshold risks returning no documents for legitimate queries with imprecise phrasing; a lower one risks filling the context with weakly relevant content that can mislead generation.

Evidence Coverage Agent

Once a response is generated, the Evidence Coverage Agent performs claim-level verification. It decomposes the generated text into atomic claims, individual factual assertions, and checks each one against the retrieved documents. Any claim that cannot be traced to the retrieved context is removed before the response is returned to the user.

This operates on a different level than the Self-RAG Critic described below. The Evidence Coverage Agent is concerned with the output: it audits what the model actually said. The Self-RAG Critic is concerned with the process: it validates each stage of the pipeline before generation proceeds. Together they enforce factual grounding from both directions.

Self-RAG Critic

The Self-RAG Critic implements a six-stage validation pipeline, modelled on the Self-RAG framework introduced by Asai et al. (2023), which demonstrated that language models can be trained to reflect on and critique their own retrieval and generation behaviour [4]. Each stage acts as a gate:

1. **Domain Relevance:** Is the query within the system’s operational scope?
2. **Retrieval Necessity:** Does the query require document retrieval, or can it be answered directly?
3. **Document Relevance:** Are the retrieved documents substantively relevant to the query?
4. **Answer-in-Context Verification:** Is the answer derivable from the retrieved context, or would the model need to draw on parametric knowledge?
5. **Support Validation:** Is each claim in the generated response explicitly supported by the context?

6. **Utility Evaluation:** Does the response actually address the user’s informational need?

A response must pass all six stages to be returned. Failure at any stage either triggers a retrieval retry, invokes the fallback mechanism, or escalates to a human agent depending on which constraint was violated.

5.2 Experimental Setup

The system was evaluated across two parallel tracks: a structured query dataset assessed through automated RAG metrics, and a functional QA test suite covering system behaviour across ten modules.

5.2.1 Dataset

The evaluation dataset comprises 211 queries distributed across four input categories: 64 English, 64 Urdu (Nastaliq script), 64 Roman Urdu, and 19 image inputs (PNG and JPG uploads processed through the OCR pipeline). The three text categories were balanced deliberately to assess cross-lingual consistency rather than aggregate performance — a system that performs well in English but degrades in Urdu would pass an aggregate threshold while failing its primary user base. Image queries were evaluated within the same pipeline run, with OCR-extracted text passed as query context before retrieval.

5.2.2 Technical Configuration

Component	Specification
Base LLM	OpenAI GPT-5
Backup LLM (English & Roman Urdu)	Llama-3.2-3B-Instruct (GGUF Quantized)
Backup LLM (Urdu)	Alif-1.0-8B-Instruct [2]
Embeddings	intfloat/multilingual-e5-base (768-dim)
Vector Store	Supabase pgvector
Knowledge Base	12+ NGO Domains (PDF, DOCX, TXT)

Table 5.1: Environment Specifications

GPT-5 served as the primary generation model for all evaluation runs. The local models (Llama-3.2 and Alif-1.0) function as offline fallbacks activated when the OpenAI API is unavailable, ensuring the system remains operational without internet access. All RAG evaluation metrics reported in Section 5.3 reflect GPT-5 outputs.

5.2.3 QA Test Suite

Functional correctness was verified through a structured QA test suite of 100 test cases spanning ten modules: RAG Pipeline, OCR and Multimodal Input, API Endpoints, Language Handling, Security and Privacy, Human-in-the-Loop escalation, Deployment and Load Testing, DeepEval Evaluation, Database Integrity, and Frontend End-to-End behaviour. Test cases were assigned priority levels (Critical, High, Medium, Low), with all Critical cases required to pass before release.

The majority of test cases passed on initial or corrected execution. The most notable failure was RAG10 (multi-turn conversation context), where follow-up queries failed to reference prior conversation turns. This was traced to the conversation memory buffer not being correctly propagated across retrieval calls and was subsequently fixed. No Critical defects remained open at the time of evaluation.

The suite validated several system properties that automated RAG metrics alone cannot capture: session isolation under concurrent load, OTP brute-force protection, agent-to-user message delivery latency, foreign key integrity in the database, and correct behaviour on invalid or adversarial inputs such as prompt injection attempts. Load testing against the 500 concurrent user target specified in the SRS was conducted using Locust against the staging environment.

5.3 Results and Performance Analysis

5.3.1 RAG Evaluation Metrics

Automated evaluation was conducted using the **Deepeval** framework across four metrics. Each metric was computed per test case with GPT-5 as the judge model, consistent with the generation model used in production.

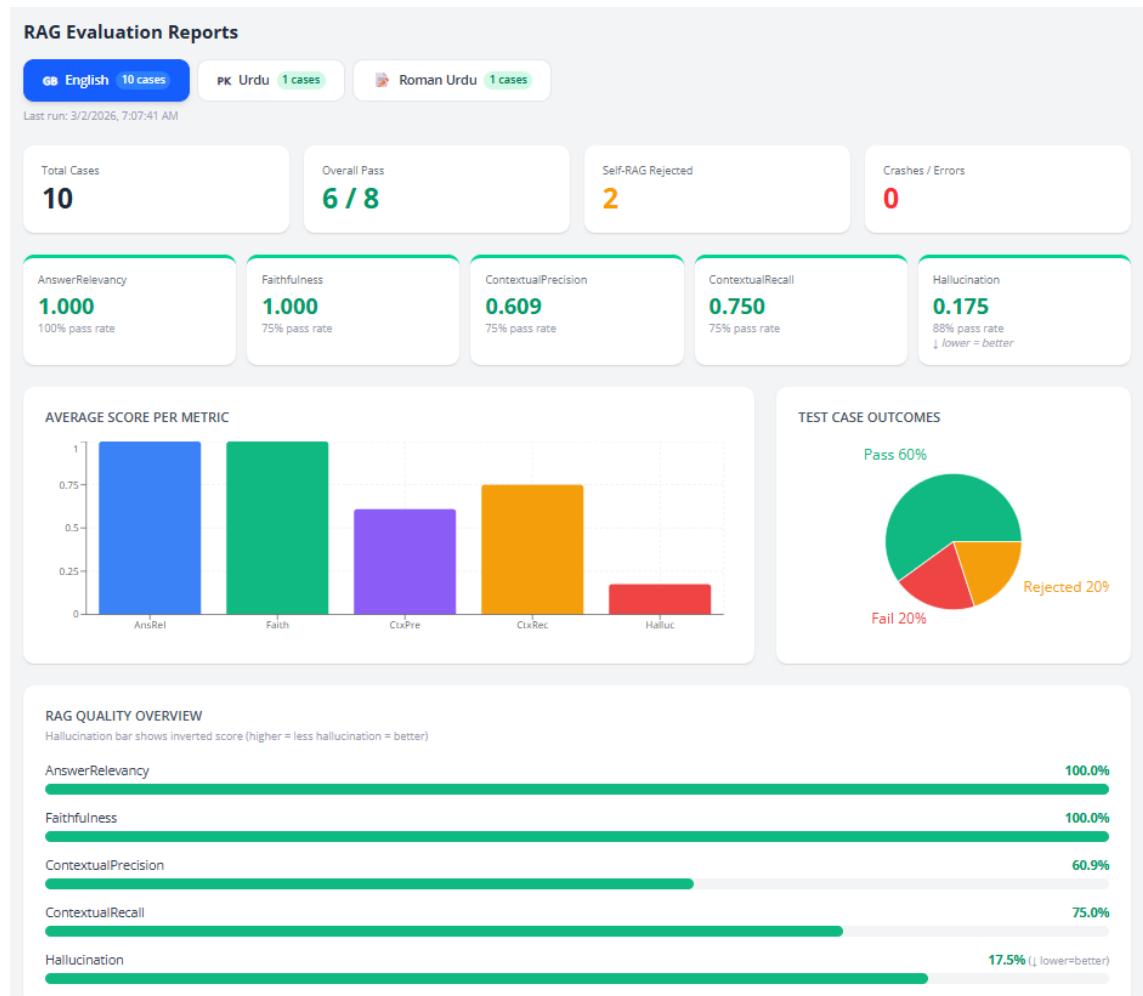


Figure 5.2: RAG Evaluation Report generated by the Deepeval framework. Scores shown are for the English evaluation suite (10 cases). Answer Relevancy and Faithfulness both achieved 1.000; Contextual Precision scored 0.609; Contextual Recall scored 0.750; Hallucination scored 0.175 (lower is better). Overall pass rate: 6/8 eligible cases.

- **Answer Relevancy** measures whether the generated response directly addresses the user’s query, penalising responses that retrieve correctly but generate tangentially. The system achieved a score of **1.000** (100% pass rate), indicating that responses consistently remained on-topic relative to the input query.

- **Faithfulness** measures whether every claim in the generated response is grounded in the retrieved context, with unsupported claims counting as hallucinations. The system scored **1.000** (75% pass rate against the threshold). The Evidence Coverage Agent’s claim-level verification and the Self-RAG Critic’s Support Validation stage are the primary mechanisms responsible for this result.
- **Contextual Precision** measures whether the retrieved documents are relevant to the query — that is, whether the retriever is returning useful chunks rather than noisy ones. The system scored **0.609** (75% pass rate), indicating that while retrieval is generally accurate, a portion of retrieved chunks carry limited relevance. This is attributable in part to the broad domain coverage of the knowledge base (12+ NGO domains), where thematic overlap between domains can cause the retriever to surface loosely related content.
- **Contextual Recall** measures whether the retrieved documents contain the information needed to answer the query — the completeness of retrieval rather than its precision. The system scored **0.750** (75% pass rate), reflecting that the fallback retrieval mechanism successfully expanded coverage for queries where the primary retrieval pass returned insufficient context.

Of the 10 English evaluation cases, 6 passed all metrics, 2 were rejected by the Self-RAG Critic as out-of-domain (a correct outcome, not a failure), and 2 did not meet the composite threshold. The hallucination score of **0.175** (lower is better; 88% pass rate) indicates that unsupported claims appeared in approximately 12% of evaluated responses — cases where the Evidence Coverage Agent’s claim removal was insufficient to fully eliminate parametric leakage from the base model.

5.3.2 Accuracy and Hallucination Control

The impact of the agentic layer on hallucination is best illustrated by the direct comparison described in Section 3.2 and shown in Figure 5.1. Without the agentic layer, the base LLM generated fabricated, contextually inappropriate content in response to a simple out-of-domain query. With the full pipeline active, generation was blocked entirely at the routing stage.

This illustrates the distinction between the two hallucination control mechanisms. The Router Agent and Self-RAG Critic prevent hallucination by blocking generation for queries the system cannot ground. The Evidence Coverage Agent addresses hallucination in responses that do proceed to generation, by removing individual claims that cannot be traced to retrieved documents. The residual hallucination rate of

17.5% reflects cases where parametric leakage passed both filters — an known limitation of claim-level verification when the base model generates plausible-sounding but unverifiable statements at the sub-sentence level.

5.3.3 Confidence Scoring and Escalation

The Confidence Scoring Engine aggregates three signals: retrieval similarity (how closely retrieved documents match the query), token probability (the model’s own certainty across generated tokens), and Self-RAG validation scores (the output of the six-stage critic pipeline). Combining these signals rather than relying on any single one reduces the risk of a high-similarity retrieval masking a low-confidence generation, or vice versa.

Queries scoring below 0.70 on the composite score are routed to human agents via the ticket escalation system. This threshold was set to prioritise user safety over system autonomy: in an NGO context where queries concern healthcare eligibility, financial aid, and beneficiary rights, a false negative (escalating a query the system could have answered) is preferable to a false positive (returning a low-confidence answer to a vulnerable user). The two Self-RAG rejections observed in the evaluation dataset demonstrate this mechanism operating as intended — the system identified out-of-domain queries and declined to answer rather than generating a plausible but unsupported response.

5.3.4 Latency and Optimization

End-to-end response time ranges from 15 to 60 seconds depending on query complexity, dialogue depth, and whether fallback retrieval or Self-RAG re-ranking is triggered. This latency is a direct consequence of three architectural decisions, each involving a deliberate tradeoff.

First, the system runs local inference for Urdu and fallback queries using quantized GGUF models (Llama-3.2-3B and Alif-1.0-8B) on CPU. Local inference eliminates API dependency and keeps the system operational offline — a meaningful constraint given the deployment context — but adds 10–30 seconds of generation time relative to a cloud API call.

Second, the Self-RAG Critic runs six sequential validation stages, each requiring a model call. This pipeline enforces factual grounding at the cost of added latency per query. Parallelising these stages is a known optimisation path but was not implemented in this version.

Third, the Evidence Coverage Agent performs claim decomposition and document-level verification after generation, adding a post-processing step before the response is returned. For complex multi-sentence responses, this step adds several seconds.

To mitigate latency, the system maintains a semantic embedding cache with a 95% similarity deduplication threshold, saving 0.5–1.0 seconds per repeated or semantically equivalent query. The Router Agent provides a further reduction by eliminating embedding computation entirely for non-informational queries, which account for an estimated 30–60% of incoming traffic.

For a production deployment, latency could be reduced substantially by moving inference to a GPU-enabled server, caching Self-RAG validation results for high-frequency query patterns, and streaming partial responses to the frontend rather than waiting for full generation to complete.

5.3.5 Multilingual and Multimodal Performance

Roman Urdu queries are translated into English before retrieval because the knowledge base is English-language. Retrieving against the Roman Urdu query directly would produce degraded similarity scores, as the embedding model’s cross-lingual alignment is stronger between Urdu Nastaliq and English than between romanised Urdu and English. Translation normalises the query into the same linguistic space as the knowledge base before the vector search runs.

Brand term protection is applied before any translation or romanisation step. Terms such as EasyPaisha, JazzCash, and Alkhidmat Foundation are wrapped in placeholder tokens, translated or romanised with those tokens intact, and restored after. Without this mechanism, translation models routinely transliterate or paraphrase proper nouns, producing outputs that fail exact string matching and corrupt domain classification.

The OCR-VLM hybrid pipeline achieved a 24.5% F1-score improvement over image-only processing for structured documents such as medical receipts and donation forms. This gain reflects PaddleOCR’s stronger handling of tabular layouts, where Tesseract alone merges or drops cell boundaries, causing downstream retrieval to operate on malformed text.

6. Conclusion and Future Work

6.1 Conclusion

The Alkhidmat Public Chat Portal successfully bridges the communication gap between one of Pakistan’s largest NGOs and its stakeholders. By transitioning from manual query handling (phone/email) to an AI-driven, multilingual RAG system, the project has achieved several key milestones:

- **Operational Efficiency:** Repetitive queries regarding blood bank locations and donation workflows are now handled autonomously, reducing staff burden.
- **Inclusivity:** The support for Roman Urdu and Urdu script makes the platform accessible to low-literacy users and the overseas diaspora.
- **Trust and Transparency:** The integration of Self-RAG verification and explicit source attribution ensures that the information provided to donors and beneficiaries is verified and grounded in official SOPs.

6.2 Future Work

While the current system provides a robust MVP (Phase 1), the following enhancements are proposed for future development:

- **Optimized Response Time:** Improving the system’s response time through optimization of the RAG pipeline and caching strategies, so system can answer queries in under 10 seconds for most cases.
- **Bilingual Voice Integration:** Implementing Speech-to-Text (STT) capabilities to allow rural beneficiaries to interact with the bot via voice notes in Urdu and regional dialects.

- **Real-time Inventory Access:** Moving beyond periodic updates to direct API integration with Alkhidmat’s healthcare and blood bank management systems for live stock availability.
- **Expanded Language Support:** Extending the NLP pipeline to include regional languages such as Sindhi, Pashto, and Punjabi to enhance localized outreach.
- **Advanced Analytics:** Implementing predictive modeling on the Admin Dashboard to forecast donation trends and healthcare service demand based on user query patterns.

7. Reflection

7.1 Reflections

7.1.1 Individual Reflections

Samah Daniyal

The most significant technical shift this project produced for me was understanding how agentic workflows fundamentally change what a language model can reliably do. Before this project, I understood RAG as a retrieval mechanism; fetch documents, pass them to a model, generate a response. What I did not appreciate was how much the model’s behaviour degrades without explicit regulation at each stage: what gets retrieved, whether it is sufficient, whether the generated output is actually grounded in it. Each component exists because the model, left to itself, will fill gaps with plausible-sounding fabrication. That understanding now shapes how I think about any LLM-based system.

The hardest challenge was not technical. Working with an external industry partner over a full academic year introduced constraints that no amount of engineering can resolve: delayed data access, shifting requirements, and a dependency on an organisation with its own priorities and timelines. Navigating that, managing expectations, adjusting scope without losing the project’s core purpose, and maintaining momentum within the team, required a different kind of problem-solving than debugging code. It is a skill I did not expect to develop through a technical FYP, and one I consider more transferable than most of what I built.

The skill I am most confident in now is debugging LLM systems at the theory level. Early in the project, encountering unexpected model behaviour felt opaque, the model produced something wrong and the cause was unclear. By the end, I could trace a failure back to its source: whether it was a retrieval threshold set too low, a prompt structure that allowed parametric leakage, or a language detection edge case the pipeline had not accounted for. That only became possible because I understood

the underlying concepts well enough to reason about them, not just run the code.

Rubab Shah

In this Final Year Project, I worked on the frontend development and the design of the agentic workflow for an AI-powered public chat portal for Alkhidmat Foundation. My role focused on creating a clean, responsive, and user-friendly interface that could serve a diverse audience, including donors, patients, and beneficiaries, while ensuring smooth integration with the backend AI system.

Alongside the frontend, I designed the agentic workflow that governs how the system processes user queries, retrieves relevant information through the RAG pipeline, and generates responses. This required understanding how to structure interactions between the user, the language model, and external data sources, as well as implementing logic for handling different query types and maintaining context.

The learning curve was significant, especially in integrating frontend components with asynchronous AI responses and in shifting from traditional programming to more dynamic, decision-based workflows. However, through continuous iteration and problem-solving, I developed a stronger understanding of full-stack AI systems.

Overall, this experience helped me improve both my technical and analytical skills, while also giving me insight into building practical AI solutions with real-world impact.

Yashfeen Zehra Zaidi

This project was a fantastic learning curve that allowed me to dive deep into the mechanics of LLMs and Retrieval-Augmented Generation (RAG) for Al-Khidmat. By implementing Agentic AI, I moved beyond simple prompts to create autonomous workflows capable of handling complex queries. I integrated confidence calculation scores to ensure accuracy, which triggers human routing whenever the AI's certainty is low, maintaining a seamless bridge between automation and manual support. Beyond these technical milestones, the experience truly drove home how much soft skills like clear communication, adaptability, and a strong work ethic impact the final result.

Fatima Dossa

The entire FYP was a rewarding experience in terms of both understanding best practices in the industry as well as from a research point of view. Collaborating with an industry partner like Alkhidmat and being in constant conversation with both supervisors offered us a unique perspective on how real projects are executed in the industry; how there are hours of project requirement meetings before finalising requirements - and how many different stakeholders are involved from developers to product leads. The research in terms of lit review for our external RAG based chatbot helped us learn what's happening recently in AI and we did this before implementing any feature from deciding how to categorise domain, confidence calculation, retrieval strategies, building the knowledge base, or chunking.

7.1.2 Team Reflection

The team comprised four members, each owning a distinct layer of the system. Samah led backend architecture and documentation. Rubab owned the frontend and the agentic module. Yashfeen managed UX, testing, and model evaluation. Fatima handled the data pipeline and image processing. This division was deliberate: the five-layer architecture — presentation, application, AI/ML, data, and infrastructure — mapped directly onto individual ownership, which reduced coordination overhead during parallel development.

The more demanding challenge was integration. Each layer made assumptions about the layers adjacent to it, and those assumptions did not always hold. The language detection module, for instance, needed to be tightly coupled to both the frontend language state and the backend retrieval strategy — a dependency that only became apparent when testing cross-lingual queries end-to-end. Resolving these integration points required sustained communication across the team rather than isolated problem-solving within each layer.

Working across different approaches to work and different tolerances for ambiguity over a year-long project is its own form of engineering. The team navigated this by maintaining shared ownership of the core evaluation criteria — if the RAG pipeline failed a test case, it was a team problem regardless of whose code was involved. That framing kept collaboration functional even when individual working styles diverged.

7.1.3 Process Reflection

What worked well. The modular architecture was the correct structural decision. Because each component — the Router Agent, the Retriever, the Evidence Coverage

Agent, the Self-RAG Critic — was implemented as an independent unit with defined inputs and outputs, individual components could be tested, replaced, and debugged without destabilising the rest of the pipeline. When the conversation memory buffer failed (RAG10), the fault was isolated and corrected without touching retrieval or generation logic. That would not have been possible in a monolithic implementation.

What could be improved. End-to-end latency of 15–60 seconds is the system’s most significant unresolved limitation. The primary contributors are sequential Self-RAG validation stages (each requiring a model call), CPU-bound local inference for Urdu queries, and post-generation claim verification. These were accepted tradeoffs given the deployment constraints — local inference was necessary to maintain offline capability — but they represent a clear target for future optimisation. Parallelising the Self-RAG validation stages, moving inference to a GPU-enabled server, and streaming partial responses to the frontend would collectively bring response times into an acceptable range for a production chat interface. The embedding cache addresses repeated queries effectively, but does not reduce latency for novel queries, which represent the harder problem.

7.1.4 Plans vs. Achievements

What was achieved. The core system — the Agentic RAG engine, the multilingual pipeline, the role-based frontend, the human escalation mechanism, and the admin analytics dashboard — was fully implemented and evaluated. The system handles English, Urdu, and Roman Urdu queries, processes image inputs through the OCR pipeline, enforces factual grounding through claim-level verification, and escalates low-confidence queries to human agents via a ticketing system. These constitute the primary deliverables of the original project proposal.

What was not achieved. WhatsApp integration was specified in the original proposal as the primary deployment channel, on the basis that Alkhidmat Foundation’s beneficiaries predominantly communicate through WhatsApp. During the project, the industry partner withdrew this requirement because they were unable to provide access to the WhatsApp Business API — a dependency outside the team’s control. The system was consequently deployed as a standalone web portal rather than a WhatsApp-native interface. This does not affect the RAG pipeline or the agentic architecture, but it does limit the system’s reach to users who access it through a browser rather than through the messaging platform already embedded in their daily use.

What was added. The agentic layer was not part of the original proposal. The initial design specified a standard RAG pipeline. During the literature review phase,

the team identified that standard RAG systems produce unacceptable hallucination rates for a deployment context involving vulnerable users and sensitive queries. The Router Agent, Self-RAG Critic, and Evidence Coverage Agent were added as architectural enhancements in response to that finding — a scope expansion that increased implementation complexity but was necessary to meet the system’s core reliability requirement.

Design priorities. The knowledge base was scoped to Healthcare and Donation domains as a deliberate MVP decision. Alkhidmat Foundation operates across 12+ service domains, and building a reliable knowledge base across all of them within the project timeline — particularly given the delays in data access — was not feasible without compromising retrieval quality in every domain. Depth in two domains was prioritised over shallow coverage across all of them. Broader domain expansion is the most immediate opportunity for future development.

8. Data

The knowledge base underpinning the system was constructed from three primary source categories, all specific to Al Khidmat Foundation’s operational domain. First, automated web crawlers extracted content from two designated official websites: the main organizational portal at *alkhidmat.org* and the Al Khidmat Health Foundation site at *ahf.com.pk*, collecting service descriptions, programme details, healthcare guidance, and donation-related information. Second, the project team manually created supplementary documents by sourcing and verifying content from these same official websites, producing informational text covering areas not sufficiently captured through scraping alone. Third, Al Khidmat Foundation directly provided a curated set of FAQs covering common public inquiries; these were ingested without modification after format normalization. Together, these sources are organized into three domains. The **Donors** domain covers financial giving to Al Khidmat Foundation, including Zakat, Sadaqah, Fidya, Fitrana, general donations, and available donation bank options. The **Healthcare** domain covers Al Khidmat’s medical service network, including hospitals, diagnostic labs, pharmacies, blood banks, ambulances, mobile health units, and countrywide facility locations. The **General** domain aggregates all remaining organizational content including volunteering, orphan support, international relief programmes, women’s philanthropy, microfinance, jobs, procurement, and media. This complete corpus forms the retrieval foundation from which all system responses are grounded.

The content from three primary sources is processed through the following stages before becoming retrievable knowledge for the system.

Data Collection

- **Web Scraping:** Automated web crawlers extracted service descriptions, programme details, healthcare guidance, and donation information from *alkhidmat.org* and *ahf.com.pk*.

- **Manual Input:** Supplementary documents were manually created by the project team by sourcing and verifying content from official websites to cover areas not captured by scraping.
- **Al Khidmat FAQs:** A curated set of FAQs covering common public inquiries was provided directly by the foundation and ingested after format normalization.

Domain Classification

Documents were categorized into three primary domains to organize the retrieval foundation:

- **Donors:** Covers financial giving, including Zakat, Sadaqah, Fidya, Fitrana, and available donation bank options.
- **Healthcare:** Covers the medical service network, including hospitals, diagnostic labs, pharmacies, blood banks, ambulances, and mobile health units.
- **General:** Aggregates content regarding volunteering, orphan support, international relief, microfinance, jobs, and media.

Data Processing and Ingestion

- **Data Cleaning:** Raw documents underwent normalization and deduplication to standardize formatting and remove redundant content.
- **Admin Document Ingestion:** Cleaned documents are managed via an admin interface, allowing authorized users to upload, update, or remove documents without manual backend intervention.
- **Preprocessing:** Documents undergo text cleaning and semantic-aware chunking into segments of 300–500 tokens.

Embedding and Indexing

Document chunks are converted into vector embeddings using the `multilingual-e5-base` model. These are indexed in a **Supabase pgvector** store with associated domain metadata to enable filtered retrieval.

Query-time Retrieval and Generation

At query time, the system performs the following steps:

- **Language Detection and Routing:** User input is detected and routed to the appropriate model:
 - **Urdu:** Routed to the **OpenAI** (with **Alif LLM** fallback).
 - **English:** Routed to **OpenAI** (with LLaMA fallback).
 - **Roman Urdu:** Routed to **OpenAI** (with LLaMA fallback).
- **Retrieval:** The query is embedded, matched against the vector store, and retrieved chunks are reranked to generate a grounded response.



Figure 8.1: Data Pipeline Architecture

References

- [1] Authors (IJARESM). *AI Chatbot Integrating Voice, Text, and Image for Enhanced Human-Computer Interaction*. 2024. URL: <https://www.ijaresm.com/ai-chatbot-integrating-voice-text-and-image-for-enhanced-human-computer-interaction#:~:text=and%20contextual%20nuances%2C%20enhancing%20the,RNNs%29%20and%20attention%20mechanisms>.
- [2] Traversaal AI. *Alif-1.0-8B-Instruct (v1.0)*. <https://huggingface.co/large-traversaal/Alif-1.0-8B-Instruct>. Large language model, accessed 2026-04-13. 2024.
- [3] W. Anwar, X. Wang, and X. Wang. “A Survey of Automatic Urdu Language Processing”. In: *Proceedings of the International Conference on Machine Learning and Cybernetics*. 2006. DOI: 10.1109/ICMLC.2006.259164.
- [4] Akari Asai et al. “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection”. In: *arXiv preprint arXiv:2310.11511* (2023). URL: <https://arxiv.org/abs/2310.11511>.
- [5] Chambers and Partners. *Data Protection & Privacy 2025: Pakistan*. 2025. URL: <https://practiceguides.chambers.com/practice-guides/data-protection-privacy-2025/pakistan#:~:text=The%20Constitution%20of%20Pakistan%2C%20under,of%20the%20European%20Union>.
- [6] *Evaluation Metrics for Retrieval-Augmented Generation (RAG) Systems*. Geeks-forGeeks. 2024.
- [7] W. Fan et al. “A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models”. In: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. 2024. DOI: 10.1145/3637528.3671470.
- [8] A. Følstad et al. “Chatbots for Social Good”. In: *Extended Abstracts of the 2018 CHI Conference on Human Factors in Computing Systems*. 2018. DOI: 10.1145/3170427.3185372.

- [9] Freshworks. *Chatbot Analytics: The Complete Guide*. 2024. URL: <https://www.freshworks.com/chatbots/analytics/>.
- [10] Y. Gao et al. “Retrieval-Augmented Generation for Large Language Models: A Survey”. In: *arXiv preprint* (2023). DOI: 10.48550/arXiv.2312.10997.
- [11] R. Gulab. *Building a Multimodal RAG Chatbot with Image, Text, and Table Understanding*. 2024. URL: <https://medium.com/@rajratangulab.more/building-a-multimodal-rag-chatbot-with-image-text-and-table-understanding-91946bc9c51c>.
- [12] A. Hakeem et al. “AI-Powered Chatbot for Conversational Understanding in Roman Urdu”. In: *International Journal of Innovations in Science & Technology* 7.6 (2025), pp. 127–138. URL: <https://journal.50sea.com/index.php/IJIST/article/view/1338>.
- [13] B. A. Halperin et al. “Probing a Community-Based Conversational Storytelling System for Advocacy”. In: *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 2023.
- [14] Heroku. *Chatbots with Heroku and Terraform*. 2024. URL: <https://www.heroku.com/blog/chatbots-with-heroku-terraform/#:~:text=The%20Live%20Agent%20team%20put, and%20responding%20to%20security%20issues>.
- [15] S. Kazi, M. Rahim, and S. A. Khoja. “Crossing Language Boundaries: Evaluation of Large Language Models on Urdu–English Question Answering”. In: *Proceedings of the First Workshop on Natural Language Processing for Indo-Aryan and Dravidian Languages*. 2025, pp. 141–151. URL: <https://aclanthology.org/2025.indonlp-1.17>.
- [16] U. Khan et al. “Urdu Natural Language Processing Issues and Challenges: A Review Study”. In: *Intelligent Technologies and Applications (INTAP 2019)*. Vol. 1198. Springer, 2020, pp. 461–470. DOI: 10.1007/978-981-15-5232-8_39.
- [17] J. Kim and H. Kwak. “Neural Conversational Agents: A Comprehensive Review”. In: *IEEE Access* (2023).
- [18] A. Kumar et al. “SAATHI: An Urdu Virtual Assistant for Elderly Aging in Place”. In: *Participative Urban Health and Healthy Aging in the Age of AI, ICOST 2022*. Vol. 13287. Springer, 2022, pp. 73–85. DOI: 10.1007/978-3-031-09593-1_6.
- [19] Capgemini Invent Lab. *Quantifying LLMs’ Uncertainty with Confidence Scores*. Medium. 2024.

- [20] P. Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020, pp. 9459–9474. URL: <https://proceedings.neurips.cc/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf>.
- [21] M. N. Maqbool, H. I. Nawaz, and S. A. Ali. “A Sentence-Level Encoder–Decoder Architecture for an Intelligent Administrative Chatbot for Roman Urdu”. In: *Applied Computational Intelligence and Soft Computing* (2025). DOI: 10.1155/acis/4728280.
- [22] AWS Marketplace. *Product View on AWS Marketplace*. 2024. URL: <https://aws.amazon.com/marketplace/pp/prodview-my257vllcsdwq#:~:text=Highlights>.
- [23] H. Mohiuddin, B. Kasi, and A. Ullah. “Multilingual Transliteration Based Urdu Chatbot Using Rasa Framework”. In: *Proceedings of the 2023 17th International Conference on Open Source Systems and Technologies (ICOSST)*. 2023. DOI: 10.1109/ICOSST60641.2023.10414237.
- [24] M. Munir et al. “Feasibility of an Artificially Intelligent Vaccines Chatbot in Pakistan: A Mixed Methods Evaluation”. In: *European Journal of Public Health* 31.Suppl 3 (2021). DOI: 10.1093/eurpub/ckab164.507.
- [25] OpenAI. *ChatGPT Can Now See, Hear, and Speak*. 2023. URL: <https://openai.com/index/chatgpt-can-now-see-hear-and-speak/>.
- [26] *Optimum Chatbot Confidence Threshold*. Cyara Blog. 2023.
- [27] “Reducing Conversational Agents’ Overconfidence”. In: *Transactions of the ACL* (2022).
- [28] D. A. Siddiqi et al. “Development and Feasibility Testing of an Artificially Intelligent Chatbot to Answer Immunization-Related Queries of Caregivers in Pakistan: A Mixed-Methods Study”. In: *International Journal of Medical Informatics* 181 (2024). DOI: 10.1016/j.ijmedinf.2023.105288.
- [29] R. Singh and A. Jain. “Persona Modeling in Conversational AI: A Survey”. In: *Artificial Intelligence Review* (2024).
- [30] John Paul Thomas. *Adding Confidence Score for LLM Results in RAG Chain Scenarios*. Medium. 2024.
- [31] “Uncertainty Quantification in Large Language Models: A Survey”. In: *ACM Digital Library* (2024).

- [32] WATI. *AI Support Agent for WhatsApp*. 2024. URL: <https://www.wati.io/ai-support-agent/#:~:text=AI%20automatically%20answers%20common%20customer,queries>.
- [33] WhatsApp. *WhatsApp Trust and Safety*. 2024. URL: <https://business.whatsapp.com/trust-and-safety/#:~:text=Every%20WhatsApp%20message%20including%20messages,delivered%20to%20the%20intended%20recipient>.